A 1D CNN for Pulse-Count Estimation in Simulated GRB Light Curves

Abstract

We describe a pipeline that (1) generates synthetic Swift/BAT-like gamma-ray burst (GRB) light curves using the existing Stern & Svensson (1996, SS96) pulse-avalanche simulator in this repository, (2) labels each simulated light curve with the number of independent parent pulses that seeded its avalanche, and (3) trains a one-dimensional convolutional neural network (1D CNN) to predict that count directly from the observed (noisy, background-subtracted) count-rate time series. We report the implementation issues that had to be resolved in the existing codebase before this was possible, the design decisions behind the labeling scheme, network architecture, and training procedure, and the evaluation methodology used to validate the trained model. Full reproduction instructions and a file-by-file description of the code are included below.

1. Background and Objective

The pulse-avalanche model reproduces the ensemble statistical properties of GRB light curves by simulating a stochastic cascade: a Poisson-distributed number of independent "parent" pulses trigger, and each recursively spawns a cascade of temporally-clustered "child" pulses (Stern & Svensson 1996; implemented in `lc_pulse_avalanche/avalanche.py`). The existing repository already provides a genetic-algorithm-calibrated parameter set that reproduces real Swift/BAT and BATSE light-curve statistics, and a script (`simulate_GRBs.py`) that draws synthetic light curves from that model.

Our objective is a supervised learning problem layered on top of that simulator: given only the observed time series of a synthetic light curve, predict how many distinguishable pulses ("peaks") it contains. Because the simulator has privileged access to the number of parent pulses that seeded each realization, it can generate an effectively unlimited, exactly-labeled training set — the central appeal of using a physically-motivated simulator as a source of ground truth for this kind of counting task.

2. Methods

2.1 Ground-truth label: number of parent pulses

The natural candidate label already present in this codebase, `n_sig_pulses` — the number of pulses judged "significant" by a MEPSA-like detectability criterion (`count_significative_pulses_ver2` in `statistical_test.py`) — could not be used: that function reads an LC attribute (`lc._ampl`) that no longer exists in the current pulse-generation code (see §3.2). We therefore used the number of

parent (spontaneous) pulses, `mu_s` in `avalanche.py`'s notation, as the training label. This quantity was already computed by the simulator on every run but was previously a local variable, discarded once the avalanche was generated.

This choice is physically motivated, not merely a workaround: each parent pulse's cascade of children is temporally clustered around its own trigger time, so in the observed (post-overlap, post-noise) light curve, one parent typically manifests as one distinguishable bump. The number of parent pulses is therefore a reasonable proxy for "the number of peaks a human, or a CNN, would count by eye" — arguably a cleaner target than a detectability threshold tuned to a specific instrument's sensitivity.

2.2 Data generation

Light curves were generated with the existing `simulate_GRBs.py config.ini` pipeline, unmodified apart from the labeling additions described in §3.1. The instrument was fixed to Swift/BAT, using the paper's existing GA-calibrated parameters (v36): `mu=1.34`, `mu0=1.16`, `alpha=2.53`, `delta1=-0.75`, `delta2=0.27`, `tau_min=0.03`, `tau_max=56.8`. No GA refitting and no real Swift/BAT catalog data were required — these parameters were already validated against real data by the repository's prior work.

Every accepted light curve (i.e., one that passed the existing T90/S-N constraint filters in `generate_GRBs()`) shares a fixed-length time grid of 15,783 bins, determined by the instrument time resolution and the `t_i` / `t_f` window hardcoded in `simulate_GRBs.py`. This was confirmed empirically (`wc -l` across generated files) rather than assumed, and it is what makes a fixed-input-length CNN workable without padding or masking logic.

Two datasets were generated for this work: an initial N=3000 set used for the first full training run, and a larger N=10,000 set generated afterward for a follow-up run at higher label-count diversity and lower sampling noise in the rarer high-pulse-count classes.

2.3 Input representation

The CNN receives the `lc` column of each exported file — the background-subtracted, noise-added count-rate series, i.e. what an instrument would actually record — rather than the underlying pulse parameter list (`lcN_lc_params.txt`). This is deliberate: the network must learn to count from the same kind of signal a real analysis pipeline would have, not from privileged access to the generative parameters.

Each light curve is normalized independently by its own peak absolute value before being passed to the network:

```
x_normalized = x / max(|x|)
```

Per-LC amplitude is set by an independent random draw (`A_max`, sampled from the instrument's peak-count-rate distribution) that carries no information about pulse count. Without this

normalization the network could partly key off absolute flux instead of waveform shape; normalizing removes that shortcut.

2.4 Network architecture

`architecture_diagram.py` renders the whole inference pipeline as a single schematic (`architecture_diagram.png`): an input light curve on the left, the five-block architecture described below in the middle, and the predicted count on the right, using a real example the model gets right (`uniform_3000`, true/predicted `n_parent_pulses` = 5, model `runs_3000_p_uni`, chosen for a matching in-distribution example rather than a mismatched natural-vs-uniform pairing).

Why PyTorch. A convolutional architecture with learnable filters and gradient-based training is not something `scikit-learn` (already a dependency of this repository) provides — it has no autodiff or convolution primitives — so a deep-learning framework was required regardless of which one. Between the two dominant options, PyTorch was chosen over TensorFlow/Keras for three practical reasons specific to this codebase: (1) its eager-execution model means the training loop in `train.py` is ordinary, readable Python — each line runs immediately and can be stepped through or printed mid-loop, which matters when debugging a repository that turned out to have this many environment/path issues (§3); (2) its `Tensor` API mirrors numpy closely, and this codebase already uses numpy throughout (`avalanche.py`, `statistical_test.py`), minimizing the new mental model needed to read `dataset.py` / `model.py`; (3) an official CPU-only wheel is distributed for Windows (`--index-url https://download.pytorch.org/whl/cpu`), matching the target machine (no GPU) without pulling in a much larger CUDA-enabled build.

The model (`PulseCountCNN` in `model.py`) is a compact 1D CNN:

Stage	Layers	Output channels	≈ Output length
Input	—	1	15,783
Block 1	Conv1d(k=15, s=2, p=7) → BatchNorm1d → ReLU → MaxPool1d(4)	16	1,973
Block 2	Conv1d(k=9, s=2, p=4) → BatchNorm1d → ReLU → MaxPool1d(4)	32	246
Block 3	Conv1d(k=5, s=2, p=2) → BatchNorm1d → ReLU	64	123
Pooling	AdaptiveAvgPool1d(1)	64	1
Head	Flatten → Linear(64→32) → ReLU → Dropout(p=0.2) → Linear(32→1)	1	—

Each row of the table is a stage the data passes through, top to bottom. **Output channels** is the number of distinct pattern-detectors ("filters") running in parallel by the end of that stage — not a physical quantity, just more independent things the network is looking for at once. **≈ Output length** is how many time-bins are left after that stage, which shrinks at every block as the network trades fine time resolution for a wider view of the signal.

In plain terms, each block is one round of "scan the signal for a pattern, then shrink it down" before the next block looks for bigger-picture patterns built from the previous ones. Taking Block 1, `Conv1d(k=15, s=2, p=7) → BatchNorm1d → ReLU → MaxPool1d(4)`, layer by layer:

- **Conv1d(k=15, s=2, p=7)** — a sliding-window filter. `k=15` (kernel size) means each filter looks at 15 consecutive time-bins at once (roughly 1 second of data, since each bin is 0.064 s) to detect a local shape, such as the rise or decay of a pulse. `s=2` (stride) means the window jumps forward 2 bins between looks instead of 1, which by itself already halves the output length. `p=7` (padding) pads 7 zero-value bins onto each end of the input beforehand — pure bookkeeping so the output length comes out to a clean, predictable size; it doesn't add or remove information.
- **BatchNorm1d** — rescales the filter's output numbers to stay in a well-behaved range. This is a standard trick for keeping training stable and fast; it doesn't change what shapes the network is capable of detecting, only how easily it learns to detect them.
- **ReLU** — a simple on/off switch: keep a number if it's positive, otherwise set it to zero. Stacking many of these across layers is what lets the network build complex pattern-detectors out of simple ones.
- **MaxPool1d(4)** — looks at every group of 4 consecutive values and keeps only the largest, discarding the other 3. This shrinks the sequence by another 4x and keeps whichever response was strongest nearby (loosely: "was there a strong version of this pattern somewhere in this neighborhood?"), which is what matters for *counting* peaks rather than knowing their exact position to the bin.

Combined, one block's stride-2 convolution and 4x max-pool shrink the sequence length by roughly 8x per block while growing the channel count — trading fine time resolution for more distinct, higher-level pattern-detectors. By the end of Block 3 the original 15,783-bin light curve has been reduced to a short sequence of 64 "was there a peak-like feature here" signals; global average pooling then collapses that down to one number per channel (64 numbers total, each summarizing that channel's average activity across the whole light curve), which the head (two small fully-connected layers) turns into the final predicted pulse count.

Global average pooling before the fully-connected head means the network's parameter count is independent of input sequence length — the same architecture accepts a different time grid (e.g. a different instrument resolution or window) without modification.

The raw scalar output is passed through `softplus(x) + 1` before being returned. `n_parent_pulses` is bounded below at 1 by construction (`avalanche.py` redraws `mu_s` until nonzero), and this activation encodes that constraint directly in the architecture rather than relying on the loss function to discourage negative or sub-1 predictions during training.

Why these components, and not alternatives?

- **1D convolution, not a fully-connected (dense) network.** A light curve is a sequence in one dimension (time only), so a 1D convolution matches its structure directly, the way a 2D convolution matches an image's two spatial dimensions. Convolution is also translation-invariant by construction: a filter that has learned to recognize a pulse's rise/decay shape

recognizes it the same way whether it occurs at t=2 s or t=80 s, without needing to see that shape at every possible position during training. A dense layer would need on the order of `15,783 × (hidden width)` independent weights just for its first layer, with no such shared structure — impractical for a training set of a few thousand examples, and blind to the fact that "a pulse near the start" and "a pulse near the end" are the same underlying pattern.

- **BatchNorm1d after every convolution.** Batch normalization is close to a default choice in modern CNNs because it stabilizes and speeds up training by keeping each layer's input distribution well-scaled. It matters somewhat more here because although each light curve is peak-normalized to its own maximum (§2.3), the *shape* of that normalized signal — how much of it sits near zero versus near the peak — still varies with pulse count and overlap, so re-centering activations layer-by-layer helps keep gradients well-behaved throughout training.
- **ReLU, not sigmoid/tanh.** ReLU (`max(0, x)`) is the standard default nonlinearity for CNNs because, unlike sigmoid or tanh, its gradient doesn't shrink toward zero for large inputs ("vanishing gradients"), letting gradients propagate through multiple stacked conv blocks during backpropagation without stalling. It is also cheaper to compute, which matters when training on CPU.
- **MaxPool1d(4) inside the first two blocks.** Two things motivate this rather than relying on the strided convolutions alone. First, practicality: the input is 15,783 bins long, and reducing it aggressively in the early blocks keeps the later, wider-channel layers cheap enough to train on CPU in reasonable time. Second, and more specific to this task: max-pooling adds a small amount of local shift-invariance — if a pulse's peak lands one or two bins to either side across two otherwise-similar light curves, max-pooling makes it more likely both produce a similar downstream signal, since what matters is *whether* a peak-like feature occurred nearby, not its exact bin.
- **AdaptiveAvgPool1d(1) at the end — average, not max, pooling.** This is deliberately different from the max-pooling used earlier. By the last block, each of the 64 channels should ideally represent something like "how much of this pattern occurred across the whole light curve." *Averaging* that channel over time produces a value that grows with how often/strongly the pattern fired, which correlates with count. *Maxing* it would instead only answer "did this pattern occur at least once, anywhere," discarding exactly the information — how many times — the network is being asked to predict. This choice is also what makes the architecture input-length-agnostic (see above).
- **Dropout(p=0.2), applied only in the head.** Placed in the small fully-connected head, not the convolutional feature extractor — the typical placement, since the dense layers have many parameters relative to the training set size (a few thousand light curves) and benefit most from this low-cost overfitting mitigation.
- **Adam optimizer, MSE loss** (§2.5). Adam's adaptive per-parameter learning rates make it a robust default that needs little manual tuning, appropriate for a first working model rather than an already-tuned one. MSE is a standard, simple regression loss; a Poisson negative-log-likelihood loss would more closely match the label's actual Poisson-distributed nature and is a reasonable next experiment, but MSE was chosen here for simplicity, leaving the positivity constraint to the softplus output above rather than the loss function.

2.5 Training procedure

The model is trained as a regression problem (continuous output, rounded to the nearest integer only at evaluation time) using mean-squared-error loss against the integer label, optimized with Adam (default learning rate `1e-3`). We chose regression over a fixed-class softmax because pulse count is an open-ended ordinal quantity with no natural upper bound to hardcode into a class list in advance.

Data are split 70% train / 15% validation / 15% test. The split is stratified by label when every class has at least two members (a requirement of scikit-learn's stratified splitting); it falls back to a plain random split otherwise, which matters for the sparsely populated high-pulse-count tail. The checkpoint saved for evaluation is the one with the **lowest validation MAE across all training epochs**, not the final epoch's weights, guarding against overfitting late in training.

2.6 Evaluation

The saved checkpoint is evaluated once, on the held-out test split (never seen during training or checkpoint selection). Three scalar metrics are reported: mean absolute error (MAE) after rounding predictions to the nearest integer; exact-match accuracy; and within-±1 accuracy (predictions off by at most one pulse). Two diagnostic figures are also produced, described in §2.7.

2.7 Validation figures

Two complementary histograms are generated by `validate.py`:

1. **True vs. predicted pulse-count distribution** (`pulse_count_distribution.png`) — grouped bars comparing, for each label value *k*, how many test-set light curves truly had *k* parent pulses against how many the model predicted had *k*. This diagnoses *distributional* bias: a model that systematically under- or over-counts a particular range shows a visible mismatch here even if its average error looks acceptable.
2. **Prediction error distribution** (`prediction_error_histogram.png`) — a histogram of `predicted - true` per light curve, centered on zero. This diagnoses *per-example* error behavior: it distinguishes a model that is consistently off by one pulse from one that is usually exact but occasionally far off, which the distribution-comparison plot alone cannot show.

Both figures use the repository's validated categorical color palette (blue `#2a78d6`, green `#008300`, fixed slot order — see the `dataviz` design skill) rather than default plotting-library colors.

2.8 Qualitative examples: individual light curves against their predictions

§2.7's histograms are aggregate diagnostics; they don't show what an individual light curve actually looks like, or which specific examples the model gets right or wrong. `prediction_examples.py` closes that gap: for any trained run, it takes one held-out test example per distinct true `n_parent_pulses` value present in that run's test split (evenly spaced across the label range if there are more distinct values than the requested example count), plots each one's normalized input time series — literally the same array the CNN receives, peak-normalized, no time axis recovery needed since that's what the model was trained and is being diagnosed on — and titles each panel with `True=k Pred=k'` in green with a checkmark if they match, red with a cross if they don't (a status encoding, not

decorative color — per the `dataviz` skill, correctness here is a state, so it also carries a symbol rather than relying on color alone). A companion `prediction_examples_table.md` lists the same information as a plain table (test-set index, true label, predicted label, correct/not).

Run against `runs_3000_p_wgt` (the "fix", §7.6) — `prediction_examples.png`, `prediction_examples_table.md`:

Test-set index	True	Predicted	Correct?
0	1	2	no
1	2	2	**yes**
3	3	2	no
4	4	2	no
30	5	3	no
215	6	2	no

Only 6 distinct true labels appear in this particular 450-example test split at all — a direct, visual confirmation of §7.2's finding that the natural (`unw`) dataset's long tail is thin enough that most classes above 5-6 pulses simply have too few held-out examples to always land in any given split. The light curves themselves corroborate §4.6's finding plotted here for real: at 1-2 pulses the individual peaks are visually distinct and countable by eye; by 3-4 pulses peaks begin to overlap into a single broader bump; by 5-6 pulses the underlying multi-pulse structure is barely distinguishable from noise-envelope shape, which is consistent with why the model's errors cluster in that direction (predicting fewer pulses than are truly present) rather than being randomly distributed.

3. Implementation issues resolved

These were pre-existing issues in the repository, unrelated to the CNN work itself, that blocked the pipeline and had to be fixed first.

3.1 `user` variable collision in `simulate_GRBs.py`

`simulate_GRBs.py` sets `user='external_user'` so that it resolves file paths relative to the repository instead of a hardcoded path belonging to one of the original authors. It also executes `from statistical_test import *`, and `statistical_test.py` independently defined its own module-level variable also named `user` (`user={'bach', 'gravity'}`, used only for that file's internal `sys.path` setup). The wildcard import silently overwrote `simulate_GRBs.py`'s `user`, so the `if user=='external_user':` branch was never taken at runtime, and the script silently fell through to a branch that ignored `config.ini` and used the wrong output path.

Resolution: renamed the internal variable in `statistical_test.py` to `_user` (a leading underscore is excluded from `import *` by Python convention). Nothing else in the codebase referenced the old name, so this is a pure namespace-isolation fix.

3.2 `n_sig_pulses` computation is broken upstream

Enabling `generate_GRBs(..., test_pulse_distr=True)` (required to compute `n_sig_pulses`) raises `AttributeError: 'LC' object has no attribute '_ampl'` inside `count_significative_pulses_ver2()`. The referenced attribute is dead code in `avalanche.py` — commented out (`# self._ampl = ampl`, line 481), left over from an amplitude-scaling scheme that was superseded by the current per-pulse `A_max` sampling approach. Reconstructing an equivalent scaling factor would require assumptions about intent that cannot be verified from the current code, so this was left unfixed and out of scope; `n_parent_pulses` (§2.1) was used instead. `test_pulse_distr` remains `False`.

3.3 PyTorch installed into the wrong Python interpreter

`conda run -n pygad3 pip install torch` reported a successful install, but `import torch` subsequently failed inside the same environment. The `pygad3` environment's own interpreter had no `pip` module at all (`python -m pip` → `No module named pip`), so the bare `pip` command resolved to a different Python installation found earlier on `PATH` (a Windows Store Python) and installed torch there instead.

Resolution: `conda install -n pygad3 pip` to give the environment its own package manager, then `conda run -n pygad3 python -m pip install torch --index-url https://download.pytorch.org/whl/cpu`, which ties the install explicitly to the target interpreter rather than trusting `PATH` resolution.

3.4 OpenMP runtime collision on import

With torch correctly installed, importing it alongside numpy raised `OMP: Error #15: Initializing libomp.dll, but found libiomp5md.dll already initialized` — a known Windows/conda packaging conflict between torch's bundled OpenMP runtime and the one numpy's MKL backend loads.

Resolution: `os.environ.setdefault('KMP_DUPLICATE_LIB_OK', 'TRUE')` is set at the top of `train.py` and `validate.py`, before `import torch`. This is the standard workaround for this conflict; it is safe in this context because training and validation run single-process on CPU with no dependence on strict OpenMP thread-affinity guarantees.

4. Code organization

4.1 Files responsible for light-curve generation

These are pre-existing repository files, not written for this work; this section describes what each one *does* in the generation pipeline, as context for the changes listed in §4.2 below.

- **`lc_pulse_avalanche/avalanche.py`** — the physical simulator core. Defines the `LC` class, whose `generate_avalanche()` method implements the SS96 stochastic process itself: draws the Poisson-distributed number of parent pulses, recursively spawns each parent's cascade of

child pulses (`_rec_gen_pulse()`), sums every pulse into a raw light curve, and applies the chosen instrument's response — effective area, background level, and a Swift- or BATSE-specific noise model — to produce the final observed count-rate series. This file has no awareness of output files, labels, or command-line arguments; it is a pure simulation library consumed by the layer below.

- **`statistical_test/statistical_test.py`** — despite its name (it is the repository's general-purpose analysis module, used for both real and simulated data), the pieces relevant to generation are: the `GRB` class, a plain container holding one light curve's time series plus its metadata (T90, pulse counts, file path, etc.); `generate_GRBs()` , the higher-level loop that repeatedly instantiates `LC` , applies the T90 and signal-to-noise constraint filters that decide whether a simulated burst is "acceptable" (`apply_constraints()`), and wraps each accepted one in a `GRB` object; and `export_grb()` / `export_lc_params()` , the two functions that serialize an accepted `GRB` to the `lcN.txt` and `lcN_lc_params.txt` files respectively.
- **`lc_pulse_avalanche/simulate_GRBs.py`** — the command-line entry point that orchestrates the two files above. It reads `config.ini` , resolves the instrument-specific physical constants (effective area, background level, S/N threshold) from the dictionaries defined in `statistical_test.py` , calls `generate_GRBs()` with the requested parameters and `N_grb` target, and creates the timestamped `simulated_LCs/<timestamp>/` output directory via `create_dir()` .
- **`lc_pulse_avalanche/config.ini`** — a plain `key = value` parameter file, not code: selects the instrument (`swift` / `batse`), the number of light curves to generate (`N_grb`), and the 7 SS96 model parameters. It carries no logic of its own; `simulate_GRBs.py` 's `read_values()` function parses it into a plain dict.

4.2 Changes made to those files

File	Change
`lc_pulse_avalanche/avalanche.py`	(a) `LC.generate_avalanche()` now stores the drawn parent-pulse count as `self._mu_s` (previously a local variable, discarded after use). Initialized to `0` in `__init__` . (b) Each entry in `_lc_params` now also records `is_parent` (True for the `mu_s` spontaneous pulses, False for anything from `_rec_gen_pulse()`) and `parent_id` (the generation-order index of its immediate progenitor; `None` for top-level parents) — used by `plot_lcs.py` (§4.5). `_rec_gen_pulse()` 's signature gained a `parent_id` parameter to thread this through the recursion.
`statistical_test/statistical_test.py`	(a) Internal `user` variable renamed to `_user` (§3.1). (b) `GRB` class gained an `n_parent_pulses` constructor argument/attribute, threaded through `.copy()` . (c) `generate_GRBs()` now passes `n_parent_pulses=lc._mu_s` when constructing each accepted `GRB` . (d) `export_grb()` writes `n_parent_pulses` as a 10th column in every `lcN.txt` . (e) `export_lc_params()` writes `is_parent` / `parent_id` as two extra columns in every `lcN_lc_params.txt` (§4.5). (f) `generate_GRBs()` 's main

File	Change
	loop prints an in-place `[simulate] cnt/N_grb` line for every accepted light curve (§4.4). (g) `generate_GRBs()` gained an optional `progress_callback` parameter, called with `(cnt, N_grb)` on every accepted light curve, for the on-screen progress window (§4.4) — `None` by default, so no other caller is affected.
`lc_pulse_avalanche/simulate_GRBs.py`	(a) `test_pulse_distr` left `False`, with a comment explaining why (§3.2). (b) `N_grb` adjusted between runs via `config.ini` — no structural change. (c) `sys.stdout.reconfigure(line_buffering=True)` added at the top so the progress prints above actually appear live in an interactive terminal (§4.4). (d) New `LiveGenerationProgress` class — opt-in (`config.ini: gui_progress = true`) on-screen progress-bar window, wired to `generate_GRBs()`'s new `progress_callback` (§4.4).
`lc_pulse_avalanche/config.ini`	(a) `N_grb` adjusted between runs — no structural change. (b) New `gui_progress` field (default `false`) toggling the on-screen progress window above.

4.3 New files (`cnn_peak_counter/`)

`dataset.py` — data loading. - `_lc_files(run_dir)`: internal helper; globs `run_dir` for `lc*.txt`, excludes the `*_lc_params.txt` companion files, and sorts numerically by the index embedded in each filename (so `lc2.txt` sorts before `lc10.txt`). - `find_latest_run(base_dir=None)`: returns the most recently modified subdirectory of `simulated_LCs/`, used as the default data source so the training scripts don't require an explicit path for the latest run. - `load_run(run_dir, normalize=True)`: the main entry point. Reads every light curve in `run_dir` with `pandas.read_csv` (whitespace-delimited, `#`-comment header), which transparently handles the literal string `"None"` in the still-unused `n_sig_pulses` column via pandas' default NA-value recognition — no special-casing needed. Returns `X` (an `(N, L)` `float32` array of the `lc` column per light curve, optionally peak-normalized per row) and `y` (an `(N,)` `int64` array of `n_parent_pulses`). Defensively pads or truncates any light curve whose length differs from the first one loaded, in case a run mixes instrument/config settings. - `cap_per_class(X, y, cap, seed=42)`: undersamples the dominant classes, keeping at most `cap` (randomly chosen) examples of each label value and every example for classes already at or below `cap`. A different, complementary strategy to `--sampler balanced` — it shrinks the training set itself rather than reweighting how existing examples are sampled (§7.10).

`model.py` — network definition. - `PulseCountCNN(nn.Module)`: the architecture described in §2.4. `forward()` accepts a `(batch, L)` tensor, reshapes to `(batch, 1, L)` if needed, and returns a `(batch,)` tensor of predicted (continuous, ≥1) pulse counts.

`train.py` — training entry point (CLI via `argparse`). - `_can_stratify(y)`: returns whether every class in `y` has at least two members, the precondition for `sklearn`'s `stratify=` argument. - `make_train_loader(X_, y_, batch_size, balanced)`: builds the training `DataLoader`; when

`balanced=True` (the default, `--sampler balanced`), uses `WeightedRandomSampler` with inverse-class-frequency weights to counter the class imbalance described in §7.3/§7.6. - `--preset {baseline,poisson-only,fix}`: shortcut that sets `--sampler` / `--loss` together, so the same three configurations can be run against any `--run-dir` (e.g. the 3,000- or 10,000-LC dataset) without remembering flag combinations (§7.10). `--cap-per-class N` applies `dataset.cap_per_class()` before the train/val/test split, for the fourth (undersampling) strategy in §7.10. - `per_class_report(y_true, y_pred_round, title)`: prints exact-match accuracy broken down by `n_parent_pulses` value — added because §7.2 showed aggregate metrics alone can hide a model that has stopped predicting rare classes entirely. - `LiveTrainingPlot`: opt-in (`--live-plot`) on-screen window showing train/val loss and val MAE/exact-match accuracy updating live, one point per epoch, in two side-by-side subplots. Needs an actual display; construction is wrapped in try/except so a missing display prints a warning and training continues without it rather than crashing (§4.4). - `evaluate(model, loader, criterion, device)`: runs inference over a `DataLoader` with gradients disabled; returns mean loss, MAE (computed after rounding and clamping predictions to ≥1), and exact-match accuracy. - `main()`: loads a dataset via `dataset.load_run`, performs the stratified 70/15/15 split (§2.5), builds `DataLoader`s, trains for `--epochs` (default 25) using the loss selected by `--loss` (`poisson` by default, §7.6), prints per-epoch and per-batch progress live (§4.4), and saves: - `best_model.pt` — the state dict with the lowest validation MAE seen (plus the input length and that MAE, for reference); - `test_split.npz` — the held-out test arrays, so `validate.py` can be run independently and repeatably without re-splitting; - `history.csv` — per-epoch loss/MAE/accuracy, for inspecting the training curve.

`validate.py` — evaluation and figure generation (CLI via `argparse`). - `plot_distribution(y_true, y_pred, out_path)`: renders the grouped-bar true-vs-predicted distribution figure (§2.7, figure 1). - `plot_error_histogram(y_true, y_pred, out_path)`: renders the signed prediction-error histogram (§2.7, figure 2). - `main()`: loads `best_model.pt` and `test_split.npz` from `--run-dir`, runs inference on the test set, prints MAE / exact-match accuracy / within-±1 accuracy plus the same per-class breakdown as `train.py`, and saves both figures alongside the checkpoint. (A peak-poor-vs-peak-rich pie chart existed briefly here and was removed — the per-class breakdown and §7.9's line-graph comparisons cover the same ground with more resolution.)

`compare_runs.py` — cross-run comparison (CLI via `argparse`). - `load_predictions(run_dir)`: loads a checkpoint + its test split and returns `(y_true, y_pred)` — the same inference logic as `validate.py`, factored out so it can be applied to more than one run. - `per_class_accuracy(y_true, y_pred, classes)`: exact-match accuracy per label value, aligned across runs that may not share the same class range. - `plot_comparison(runs, out_path, title)`: line graph of per-class accuracy, one line per run, over a shared x-axis of `n_parent_pulses` — used to compare sampler/loss configurations (§7.6) or dataset sizes (§7.7) directly rather than only via side-by-side tables. Takes any number of labeled runs via repeatable `--run run_dir:Label` arguments.

`compare_fix_effect_by_size.py` — cross-size comparison of the §7.6 fix itself (not just accuracy). `fix_effect(baseline_dir, fix_dir)` computes per-class `(balanced accuracy) − (uniform accuracy)`; `main()` computes this at both N=3,000 and N=10,000 and plots both as a line graph, to see whether the fix's benefit grows, shrinks, or reverses with more data (§7.9). Hardcoded to the four

specific run directories from this project rather than taking `--run` arguments, since it compares a fixed pair of pairs rather than an arbitrary set of runs.

`REPORT.md` — this document.

4.4 Live progress reporting

Both light-curve generation and training can run for tens of minutes to hours at the larger dataset sizes in §7. Two issues had to be fixed before either showed any progress before completion:

- **No progress output at all during generation.** `generate_GRBs()`'s main loop (§4.1) previously printed nothing until every requested light curve was done. It now prints an in-place-updating `[simulate] cnt/N_grb` line (a carriage-return update — `\r` plus `end=''`, right-padded to `N_grb`'s digit width — not one line per LC) every time a light curve is accepted, followed by a final newline once generation finishes so subsequent output starts cleanly. `train.py` similarly prints a `batch/n_batches` line four times per epoch, in addition to its existing per-epoch summary.
- **Output buffering.** By default, Python buffers `stdout` in blocks (rather than flushing per line) whenever it isn't connected to an interactive terminal. `sys.stdout.reconfigure(line_buffering=True)`, added near the top of `simulate_GRBs.py`, `train.py`, and `validate.py`, forces a flush after every line/carriage-return so the progress above updates immediately **in an interactive terminal you are actively running the command in** (§5.2) — this is the setup to use if you want to watch progress live.

This does *not* reliably help when the process is instead launched as a detached background job whose output is redirected to a log file (e.g. via a job scheduler, or `Start-Process` / `&`-backgrounding) — in that setup, something in the redirection/capture path can still withhold output until the process exits regardless of the script's own flushing, and the exact behavior depends on how the backgrounding is done. If you need to monitor a background run, prefer watching the growing count of `lcN.txt` files in the output directory over tailing a redirected log.

On-screen (graphical) progress, opt-in and separate from the terminal output above:

- **During generation** (`config.ini: gui_progress = true`): `simulate_GRBs.py`'s `LiveGenerationProgress` opens a small window with a horizontal bar that fills from 0 to `N_grb` as light curves are accepted, wired via `generate_GRBs()`'s new `progress_callback` parameter. The window stays open (blocking, via `plt.show()`) once generation finishes, so the final "100% done" state is visible until you close it.
- **During training** (`train.py --live-plot`): `LiveTrainingPlot` opens a window with two side-by-side subplots — train/val loss, and val MAE/exact-match accuracy — each updating with one new point per epoch. Also stays open (blocking) after training completes, showing the full curve for review.
- Both are opt-in (off by default) because they need an actual display — running them via a background/redirected process, or on a machine with no GUI toolkit available, would either show nothing or fail outright. Both wrap their window-creation in `try` / `except`: if no display

is available, they print a warning and the run continues normally with only the terminal progress from above — a missing display is a degraded experience, not a crash.
- Both use a `QtAgg`-backed interactive `matplotlib` window (confirmed available in this project's environment, with `tkinter` as a fallback toolkit); no new dependency was added; the same `matplotlib` already used for the histograms elsewhere in this project.

4.5 Light-curve visualization (`plot_lcs.py`), and two findings from building it

`lc_pulse_avalanche/plot_lcs.py` (new) plots each `lcN.txt`'s observed time series and saves `lcN_plot.png` alongside the existing `.txt` files in that run's directory. Usage:

```
 python plot_lcs.py                                    # plots the most recent run
python plot_lcs.py --run-dir ../simulated_LCs/<timestamp>
python plot_lcs.py --limit 20                          # only plot the first N (large
runs)
```

Building this required the simulator to record information it previously discarded: `avalanche.py`'s `_lc_params` entries (§2.1, §4.1) gained two new fields — `is_parent` (True for the `mu_s` spontaneous pulses, False for anything spawned by `_rec_gen_pulse()`) and `parent_id` (the index, in generation order, of each pulse's *immediate* progenitor; `None` / `-1` for top-level parents). `export_lc_params()` now writes both as extra columns in `lcN_lc_params.txt`. `plot_lcs.py` uses these to mark parent pulses (numbered, red) and child pulses (small, muted) at each pulse's peak time, placed on the observed curve at the nearest time bin. (Datasets generated before this change lack these two columns; `plot_lcs.py` detects this and skips the markers rather than erroring.)

Building and using this tool surfaced two things worth recording:

Finding 1 — the recursion's time bookkeeping is correct, not the bug the code comments worried it might be. `_rec_gen_pulse()`'s docstring (`avalanche.py:201-220`) contains a debate, apparently between the original authors, over whether each pulse's time delay (`t_shift`) is correctly accumulated across generations or incorrectly reset relative to just the immediate parent. This was checked empirically rather than by reading code alone: across all 22,609 child pulses in a 20-LC test batch, every single one had a `t_delay` strictly later than its immediate progenitor's — zero violations. The recursion does correctly carry the cumulative absolute time forward through arbitrarily many generations, exactly as the comment concludes it should. One direct consequence, visible once `plot_lcs.py` could show it: a long-lived cascade (one where `tau` shrinks slowly across generations before finally dying out) can legitimately drift its descendants far in time from their own originating parent — visually, a child pulse can end up closer to a *different* parent's cluster than its own. Combined with each pulse's amplitude being drawn independently of its generation depth (so a late descendant can be brighter than its own parent), this fully explains why a "child" marker can appear somewhere unintuitive without any bug being involved.

Finding 2 — `n_cut=2500` (`simulate_GRBs.py:155`) is a real, frequently-hit constraint, not a theoretical edge case. This cap on total pulses is shared across an *entire* light curve's avalanche (every parent's cascade combined), checked via `if self._n_pulses < self._n_cut` before extending any branch further. Tested on 100 freshly generated LCs: 13 (13%) hit or exceeded the cap, with

values clustering tightly just above 2,500 (2507-2535) — confirming genuine truncation rather than coincidence — and another 5 (18 total) were within striking distance (≥2,000). Mean `n_pls` across the 100 was ~699, but the distribution is heavily right-skewed: most light curves have well under 100 total pulses, while a minority spike into the thousands. Because the budget is shared globally rather than per-parent, one prolific family can truncate a different parent's cascade within the same light curve — a real, if secondary, source of the kind of "why does this family look artificially sparse" discrepancy discussed above. This has not been changed; it is the existing, working behavior of a repository file not otherwise touched by this project, documented here because it was surfaced by building the visualization tool, not because it was identified as something requiring a fix.

Observation 3 (qualitative, not yet quantitatively verified) — dense clusters of child pulses tend to visually coincide with what a human would call "a peak," more so than the `is_parent`-tagged pulse itself. Reviewing `plot_lcs.py`'s output revealed that the region a human eye would identify as the light curve's dominant bump is often where many child pulses (from one or several families) pile up in time, rather than necessarily sitting exactly on a numbered parent marker. This connects directly to §2.1's original justification for using `n_parent_pulses` as a proxy for "number of visually distinguishable peaks": that argument assumed each parent's cascade produces roughly one bump centered near the parent. This observation suggests the correspondence may be less clean than that — a human counting bumps by eye could plausibly count *cluster regions* (which can pool children from multiple families, per Finding 1's time-drift, or split one family's cascade across a wide span) rather than *parents* one-for-one. This has not been quantified — doing so would mean comparing `n_parent_pulses` against an actual independent peak-counting method (e.g., a real peak-finding algorithm run on the observed curve, which is also what the broken `n_sig_pulses`/MEPSA-style path in §3.2 was originally meant to provide) rather than against visual impression alone. Until that comparison exists, this is a flagged, plausible limitation on §2.1's proxy argument, not a confirmed discrepancy — recorded here because it surfaced directly from building the visualization tool, the same way Findings 1 and 2 did.

Finding 4 — a genuine contradiction between two comments in the pre-existing codebase about what units the `lc` column is in. `avalanche.py:506` states explicitly: *"For Swift, the variable `_plot_lc` contains the COUNT RATES (and not the counts!)"*. `statistical_test.py:3185` says the opposite when constructing the `GRB` object: `counts=lc._plot_lc,

these are COUNTS (not count rates!)`. Tracing the actual computation

resolves this: `self._model_rate = self._model / self._res` (counts divided by the time resolution) is what feeds `_plot_lc` for Swift, which is a rate (counts/second) by construction, not a count — `avalanche.py`'s comment is correct, and `statistical_test.py`'s is wrong (most likely a copy-paste artifact from the BATSE branch immediately above it, where `_plot_lc` genuinely is in counts). This means `plot_lcs.py`'s y-axis was mislabeled "Counts" and has been corrected to "Count rate [cnt/s]"; no other part of this project's pipeline treats the `lc` column's units in a way that this affects (the

CNN's input is peak-normalized per light curve, §2.3, so it's insensitive to which unit convention the raw signal is in).

4.6 How does the number of parent pulses actually change the light curve?

Comparing real generated examples across `n_parent_pulses` = 1, 2, 3, and 4 (`simulated_LCs/2026-07-22_13-57-00/lc*_plot.png` and `simulated_LCs/2026-07-21_09-13-51/lc9_plot.png`) makes the effect concrete rather than abstract:

- **More parents → more distinct active regions ("bumps"), spread further across the light curve.** A 1-parent light curve is a single cluster of activity around one point in time. By 4 parents (`lc9.txt` , count rate peaking at ~28 cnt/s around t≈130s), the light curve shows visibly separate clusters around t≈0-20s, t≈60-90s, t≈100-160s, and t≈220-260s — each traceable to a different parent's cascade (§2.1, §4.5).
- **The active window widens, but the total light curve length doesn't.** Every light curve, regardless of `n_parent_pulses` , is generated on the same fixed -10s to +1000s grid (§2.2) — what changes is how much of the early part of that window (roughly the first 0-300s here) contains signal versus pure background noise. More parents means a longer *stretch* of real activity, not a longer *file*.
- **Peak brightness tends to increase with more parents, but not linearly or reliably** — each pulse's amplitude is drawn independently (§2.4, §7.10's `norm` discussion), so a 4-parent light curve has more *chances* for a high-amplitude pulse to occur, but a 1-parent light curve can still, by chance, be brighter than a 4-parent one. Compare `lc9.txt` 's peak (~28 cnt/s, 4 parents) against typical 2-3 parent examples in the same batch, which mostly peak lower — a tendency, not a guarantee.
- **The relationship is not "N cleanly-separated bumps for N parents."** Per §4.5's Findings 1 and Observation 3, cascades can drift in time and interleave with each other, and dense child-pulse clusters often read as "one visual peak" regardless of how many parents contributed to them. So while the *general* trend (more parents → busier, longer-active light curve) is visually obvious and consistent across every example examined, a precise one-to-one "count the bumps, get `n_parent_pulses` " reading is not reliable — which is exactly the difficulty the CNN in this project is up against (§7.2-§7.11).

This is also the mechanism the `uni` dataset (§7.11) exploits: since raising `mu0` shifts the *typical* number of parents upward (§7, this methodology also underlies how `generate_uniform_bins.py` targets each bin), it also shifts the typical light curve toward "busier, more spread-out" — the visual pattern above, not just an abstract label change.

5. Reproduction guide

These steps reproduce the pipeline from a clean checkout, on Windows with conda (as used for this work); adapt paths/activation commands for other platforms.

Every cross-run comparison chart and table referenced anywhere in this report is collected in one place: `cnn_peak_counter/all_comparisons/` (copies — the originals stay in

whichever run directory produced them). Single-run diagnostics (`pulse_count_distribution.png` , `prediction_error_histogram.png`) are not duplicated there; those live only in their own run's directory since they're specific to one model, not a comparison.

5.1 Environment setup

```
 # Create the environment (only if it doesn't already exist)
conda create --override-channels -c conda-forge -n pygad3 python=3.10 -y
conda activate pygad3
conda install -n pygad3 -y --override-channels -c conda-forge ^
    numpy==1.26.4 matplotlib==3.8.3 pandas==2.2.0 astropy==6.0.0 ^
    pygad==3.3.1 tqdm==4.66.2 scipy==1.12.0 cloudpickle==3.0.0 ^
    h5py==3.10.0 astroml==1.0.2.post1 scikit-learn==1.4.1.post1

# The env needs its own pip (see 3.3) before installing torch
conda install -n pygad3 -y --override-channels -c conda-forge pip
conda run -n pygad3 python -m pip install torch --index-url
https://download.pytorch.org/whl/cpu
```

5.2 Generate a labeled dataset

Edit `lc_pulse_avalanche/config.ini` : - `instrument = swift` (or `batse`) - `N_grb = <desired count>` (3000 took ~20 minutes on the machine this was developed on; scale roughly linearly)

Then, from `lc_pulse_avalanche/` , either run it non-interactively via `conda run` :

```
conda run -n pygad3 python simulate_GRBs.py config.ini
```

or, in an interactive terminal you're actively watching (recommended if you want to see the live progress described below), activate the environment once and run it directly:

```
 cd C:\Users\chris\OneDrive\Documents\SyntheticLCs\lc_pulse_avalanche
conda activate pygad3
python simulate_GRBs.py config.ini
```

Live progress: `generate_GRBs()` prints an in-place-updating `[simulate] cnt/N_grb` line (carriage-return update, not one line per LC) as each light curve is accepted, so you can watch progress and estimate time remaining without waiting for the run to finish. This only updates visibly in a real interactive terminal — output redirected to a file (as happens when a process is run in the background) won't show it updating live.

This creates `simulated_LCs/<timestamp>/` , containing `lcN.txt` (time series + labels, including `n_parent_pulses` as the last column) and `lcN_lc_params.txt` (raw per-pulse parameters; not used by the CNN) for each accepted light curve, plus a copy of the config used (`config_<timestamp>.txt`).

5.3 Train

From `cnn_peak_counter/`:

```
conda run -n pygad3 python train.py --run-dir "../simulated_LCs/<timestamp>"
# or, to use whichever run directory was generated most recently:
conda run -n pygad3 python train.py
```

Useful flags: `--epochs`, `--batch-size`, `--lr`, `--val-frac`, `--test-frac`, `--seed`, `--out-dir` (default `./runs`). When comparing multiple dataset sizes, as in §7, pass a distinct `--out-dir` per run (e.g. `--out-dir ./runs_10000_p_wgt`) so each run's checkpoint, test split, and history are kept separate rather than overwriting the previous run's.

5.4 Validate

```
conda run -n pygad3 python validate.py --run-dir ./runs_3000_mse_unw
```

Prints MAE / exact-match / within-±1 accuracy to the console, and writes `pulse_count_distribution.png` and `prediction_error_histogram.png` into `--run-dir`.

6. Expected effect of a larger dataset (N=10,000)

A second dataset of N=10,000 light curves was generated (same instrument and parameters as §2.2) alongside this report, to be trained on separately. Ahead of those results, it is worth being explicit about what scaling the dataset by roughly 3.3x should and should not be expected to change, based on general supervised-learning behavior rather than results specific to this problem:

- **The label distribution keeps the same shape.** `mu0=1.16` is fixed across every generation run, so the underlying Poisson-shaped label distribution (mostly 1-2 parent pulses, tailing off sharply by 5-6) doesn't change with N — a larger dataset gives more *absolute* examples of each class, not a more balanced *proportion* of them. The rare high-count tail (5, 6+ parents) remains a small fraction of the dataset either way.
- **The rare classes should benefit disproportionately.** A 50-LC pilot sample showed classes like "5 parents" or "6 parents" with single-digit example counts at N≈3000-scale — too few for the model to learn a reliable pattern for, and too few in the test split to measure accuracy on with any confidence. At N=10,000 those same rare classes scale up roughly proportionally in absolute count, which should improve both the model's ability to learn them and the reliability of the reported per-class accuracy. This is where the largest qualitative improvement is expected — more than in the already well-populated 1-2 parent classes.
- **Held-out metrics should become less noisy.** MAE and accuracy computed on a 15%-of-3000 ≈ 450-example test set carry more sampling variance than the same metrics on a 15%-of-10,000 ≈ 1,500-example test set. Part of any difference between the two runs' reported numbers should be attributed to this effect rather than assumed to be a genuinely better model.
- **More optimizer steps per epoch.** With a fixed batch size, one epoch over 10,000 light curves is ~3.3x more gradient updates than one epoch over 3000. Comparing training curves

between the two runs should account for this — it is not purely a "more data" effect.
- **Diminishing returns should be expected past some point.** This is a deliberately compact model (§2.4) relative to the input size; more data mostly helps a model like this by reducing overfitting/variance on classes it can already represent, rather than unlocking qualitatively new capacity. If accuracy on the rare tail is still poor at N=10,000, that would point toward a class-imbalance problem — better addressed by oversampling rare classes, or varying `mu0` across generation runs to deliberately produce more high-count examples — rather than a needs-more-data problem, and would be worth investigating before simply scaling N further.
- **Generation and training time scale roughly linearly with N.** Light-curve generation took approximately 20 minutes for N=3000 on the development machine, so N=10,000 was expected to take on the order of an hour; each training epoch likewise takes roughly 3.3x longer at N=10,000 than at N=3000 for the same batch size.

The actual comparative metrics and both validation figures for the N=10,000 run will be appended here once training on it completes.

7. Results

Training configurations tested. Every run directory follows `runs_{N}_{loss}_{weighting}`, where `loss` is `mse` or `p` (Poisson NLL, §2.5) and `weighting` is one of four strategies for handling the class imbalance in `n_parent_pulses` (§7.3):

- `unw` (unweighted) — the natural, Poisson-imbalanced dataset, plain random sampling, no reweighting of any kind.
- `wgt` (weighted) — the same natural dataset, but training batches are drawn via `WeightedRandomSampler` with inverse-class-frequency weights (§7.6) — no data is removed, rare examples are just shown more often.
- `cap` (capped) — the same natural dataset, but undersampled to at most 1,000 examples per class *before* training (§7.10) — majority-class data is permanently discarded; rare classes are untouched; sampling during training is then plain/unweighted on this smaller set.
- `uni` (uniform) — not a resampling of the natural dataset at all, but a separately-generated dataset built by running the simulator at a different `mu0` per target bin and harvesting only the matches (§7.11), every other Bazzanini-calibrated parameter held fixed. Capped below the full 1-10 range originally planned, and by a different amount at each size, because the same per-bin cost growth documented in §9's diagnostic keeps biting harder the more matches are needed per bin: **300 light curves for each of bins 1-8** at N=3,000 (2,400 total), but only **1,000 light curves for each of bins 1-7** at N=10,000 (7,000 total) — bin 8 was dropped at the larger size after its cost trend (bin 7 alone slowed to roughly a quarter of bins 1-6's pace) made finishing it impractical against the deadline.

Every named condition, in plain terms:

Condition	Loss	Data / sampling	In one sentence
`mse_unw`	MSE	Natural data, plain random sampling	The original, uncorrected baseline — trains on the imbalanced data exactly as generated, no attempt to counter the imbalance.
`p_unw`	Poisson NLL	Natural data, plain random sampling	Same as `mse_unw`, but with a loss function suited to count data — isolates whether the loss alone (with no resampling) fixes anything.
`mse_wgt`	MSE	Natural data, `WeightedRandomSampler`	Rare classes are shown to the model more often during training (no data removed) — the reweighting-alone condition, MSE side.
`p_wgt`	Poisson NLL	Natural data, `WeightedRandomSampler`	The original "fix" (§7.6): reweighting plus a count-appropriate loss, on the theory both should help.
`mse_cap`	MSE	Natural data, undersampled to ≤1,000/class	Majority classes are permanently thinned before training so no class has more than 1,000 examples; sampling is then plain/unweighted on this smaller, flatter set.
`p_cap`	Poisson NLL	Natural data, undersampled to ≤1,000/class	Same undersampling, Poisson loss — isolates whether capping's effect (where it has one) depends on the loss function.
`mse_uni`	MSE	Separately-generated, genuinely flat dataset (§7.11)	Not a resampling trick at all — the dataset itself has equal examples per bin from the start, so there is nothing left to reweight or cap.
`p_uni`	Poisson NLL	Separately-generated, genuinely flat dataset (§7.11)	Same flat dataset, Poisson loss — the one comparison in this whole report where the loss function is the *only* variable left, since the data needs no correction.

That is a 2×4 factorial (loss × weighting strategy) at each of the two dataset sizes tested, 16 conditions in total:

Weighting → / Loss ↓	`unw`	`wgt`	`cap`	`uni`
`mse`	✅ done (both N)	✅ done (both N)	✅ done (both N)	✅ done (both N)
`p` (Poisson)	✅ done (both N)	✅ done (both N)	✅ done (both N)	✅ done (both N)

`mse_unw` and `p_wgt` are the original "baseline" and "fix" from §7.2/§7.6; the other six were added in §7.10/§7.11 to isolate exactly which of {loss function, sampler reweighting, data quantity, data composition} is responsible for what.

7.1 Comparison across dataset sizes

Run	N	Sampler / loss	Train/val/test split	Test MAE	Exact-match accuracy	Within-±1 accuracy	Status
`runs_3000_mse_unw` (baseline)	3,000	uniform / MSE	2,100 / 450 / 450	0.633	46.2%	92.2%	Complete (§7.2)
`runs_3000_p_wgt` (fix)	3,000	balanced / Poisson	2,100 / 450 / 450	0.749	42.7%	86.0%	Complete (§7.6)
`runs_10000_mse_unw` (baseline)	10,000	uniform / MSE	7,000 / 1,500 / 1,500	0.670	43.7%	91.6%	Complete (§7.9)
`runs_10000_p_wgt` (fix)	10,000	balanced / Poisson	7,000 / 1,500 / 1,500	0.750	39.5%	87.9%	Complete (§7.7)
`runs_20000`	20,000	balanced / Poisson	~14,000 / 3,000 / 3,000	—	—	—	On hold — not started

The second row isolates the effect of the §7.6 fix alone (same data, same split, same seed as the baseline); the third and fourth rows are the same isolation repeated at N=10,000 (§7.9); the fifth row is on hold pending a decision on whether it's needed given the §7.7/§7.9 findings.

7.2 What happened at N=3,000

The full label distribution generated at N=3,000 was:

`n_parent_pulses`	Count	Share
1	1,220	40.7%
2	1,089	36.3%
3	487	16.2%
4	157	5.2%
5	39	1.3%
6	8	0.3%

The trained model reached a held-out test MAE of **0.633**, exact-match accuracy of **46.2%**, and within-±1 accuracy of **92.2%**. Those top-line numbers look reasonable in isolation, but the two validation figures (`runs_3000_mse_unw/pulse_count_distribution.png` , `runs_3000_mse_unw/prediction_error_histogram.png`) show what's actually happening underneath them: **the model essentially stopped predicting anything other than 1 or 2.** The true test-set distribution has real mass out to 6; the predicted distribution has none above 2 — every light curve truly labeled 3, 4, 5, or 6 was predicted as 1 or 2. This is visible directly in the error histogram: the negative-error bars (predictions falling short of the true count) account for essentially the entire non-zero-error mass, with almost no over-prediction.

This also explains the apparent tension between the metrics: MAE and within-±1 accuracy look good mainly *because* 77% of the test set truly has only 1-2 parent pulses, so "always guess 1 or 2" is a low-

regret strategy under a mean-error metric even though it never engages with the actual counting task for a quarter of the data. Exact-match accuracy (46.2%) is the metric least flattered by this behavior, and it's correspondingly the weakest number. The per-epoch training history (`runs_3000_mse_unw/history.csv`) is consistent with this reading: training loss fell steadily for all 25 epochs (0.924 → 0.807), while validation accuracy barely moved (0.362 → 0.420) and plateaued early — the network kept improving its ability to fit the dominant classes rather than learning to discriminate the rare ones.

7.3 Why this happened

Four factors plausibly contributed, roughly in order of expected impact:

1. **Class imbalance.** Only 8 of 3,000 light curves (0.3%) have 6 parent pulses; only 39 (1.3%) have 5. After a 15% test split, that's ~1 and ~6 examples respectively — too few for the model to ever receive a strong, consistent training signal for those classes, and too few to reliably *measure* performance on them even if it had learned something.
2. **MSE regression pulls toward the conditional mean.** With squared-error loss, the gradient contribution from the numerous, easy 1-2-pulse examples dominates the total loss simply by weight of numbers, pushing the model toward always predicting close to the population mean (≈1.9) rather than risking a confident but occasionally wrong extreme prediction. This is a well-known failure mode of plain MSE regression on imbalanced/skewed count data, independent of network architecture.
3. **Possible intrinsic signal ambiguity.** Parent-pulse cascades can overlap heavily in time, especially for small `tau` draws (§2.4 of the underlying SS96 model allows child pulses within `alpha * tau0` of their parent). Two, three, or more overlapping cascades can plausibly blend into a light curve that doesn't look very different, post-noise, from one with fewer, cleaner cascades. Some part of the ceiling here may be information the noisy observed signal genuinely does not preserve cleanly — not solely a data-quantity or loss-function problem. This cannot be distinguished from factors 1-2 without further experiments (§7.5).
4. **Model capacity and information loss from global pooling.** The network is deliberately compact (§2.4), and global average pooling discards all positional information before the final prediction — the head sees only "how active was each channel on average," not "how many separate activation events occurred." That representation may simply cap out before it can resolve fine-grained count differences in the tail.

7.4 Why real Swift/BAT light curves should resemble these simulations, despite using no real catalog data directly

It's worth being precise about *why* this pipeline can claim any resemblance to real data at all, since the immediate generation step (§2.2) never touches a real GRB. The seven SS96 parameters used (`mu=1.34, mu0=1.16, alpha=2.53, delta1=-0.75, delta2=0.27, tau_min=0.03, tau_max=56.8`) are not arbitrary or hand-picked — they are the published output of a genetic-algorithm optimization (the `geneticgrbs.py` / `v36 (bazzanini+24 paper)` fit already present in this repository) that searched parameter space to *minimize the statistical distance between simulated and real Swift/BAT light curves* — matched on ensemble properties such as peak-count distributions,

autocorrelation function shape, and T90 distributions, against the real Swift/BAT catalog. That fitting step is where the real data enters the pipeline; we did not have to touch the catalog again downstream because its statistical fingerprint is already compressed into those seven numbers. Drawing new synthetic light curves from that calibrated regime therefore reproduces the same *population-level* statistics as real Swift/BAT bursts, by construction — that is the entire purpose of the original calibration, and it's why no GA refitting or real-catalog access was needed for this work (as established earlier in this conversation).

This resemblance, however, is real only at the **ensemble/statistical** level, and it is important not to overstate it:

- It says nothing about whether any *individual* simulated light curve resembles any *individual* real GRB — only that the two populations' aggregate statistics (the ones the GA optimization targeted) match.
- Most importantly for this specific model: `n_parent_pulses` **has no real-data counterpart.** It is a latent quantity internal to the SS96 generative process — a real GRB has no directly observable "number of independent trigger events," only an observed light curve. There is no way to obtain a ground-truth `n_parent_pulses` label for a real Swift burst to check the trained model against; §2.1's physical argument for why it's a reasonable proxy for "visually distinguishable peaks" is exactly that — an argument, not something verified against real data here. This is the most important caveat on the entire exercise and is restated in §8.
- The calibration is specific to the ensemble Swift/BAT was fit against (long GRBs, the `t90_threshold` / `sn_threshold` selection already baked into `generate_GRBs()`, §2.2's constraint filters). It does not guarantee the same closeness for GRB subpopulations outside that selection.

7.5 What can be done about the mode-collapse behavior

In roughly increasing order of implementation effort:

- **Stratified/weighted batch sampling.** Currently only the train/val/test *split* is stratified (§2.5); minibatches themselves are drawn uniformly at random, so a batch is still dominated by 1-2-pulse examples. A weighted sampler (e.g. `torch.utils.data.WeightedRandomSampler`, inverse-frequency weights) would give rare classes equal presence within every batch, directly countering factor 1 in §7.3 without needing more data.
- **A loss function suited to imbalanced count data.** Plain MSE is what drives the mean-seeking behavior in §7.3. A Poisson negative-log-likelihood loss would match the label's actual generating distribution; alternatively, per-class-weighted MSE (upweighting rare classes' contribution to the loss) is a smaller change with a similar effect.
- **Targeted data generation.** Rather than only increasing N at the existing `mu0=1.16`, generating additional batches at deliberately higher `mu0` would produce many more naturally-occurring high-pulse-count examples to mix into training, directly enriching the rare classes instead of waiting for them to accumulate at their natural (low) rate.
- **More data at the existing distribution** — the N=10,000/20,000 runs already underway. Expected to help (§6) but, per §7.3's point 3, cannot by itself resolve the issue if part of the

ceiling is genuine information loss in the observed signal rather than a shortage of examples — this is exactly what comparing 3,000 vs. 10,000 vs. 20,000 is designed to reveal (§7.6): if the tail accuracy improves substantially with N, the bottleneck is data/imbalance; if it stays flat, the bottleneck is more likely information loss or model capacity.
- **Architecture changes**, if the above don't resolve it: replacing global average pooling with a representation that retains some positional/count structure (e.g. summing per-channel activations above a learned threshold, closer to a soft peak-counting operation, rather than averaging), or simply a deeper/wider network — though this is the most expensive lever to pull and is not indicated as the first fix given factors 1-2 are cheaper to rule out first.

7.6 Chosen fix: balanced sampling + Poisson loss (implemented)

We evaluated the §7.5 options against three criteria: (1) can it be tested immediately, without waiting on new data generation; (2) does it directly target the mechanisms identified in §7.3, rather than just working around them; (3) implementation cost. Targeted data generation and "more data" both leave the core mechanism untouched — they supply more examples but don't change how those examples are weighted during training (§7.3, points 1-2) — and architecture changes are the most expensive option and, per §7.5, not indicated as a first move. That leaves the two cheapest, most directly targeted fixes: weighted batch sampling and a count-appropriate loss. These are not mutually exclusive and address different mechanisms (representation *during* training vs. how errors are *penalized*), so both were implemented together:

1. **Balanced sampling.** `train.py`'s new `make_train_loader()` builds training batches with `torch.utils.data.WeightedRandomSampler`, weighting each example by the inverse frequency of its class (`1 / count(class)`), sampled with replacement. Every training batch now has, in expectation, roughly equal representation across `n_parent_pulses` values, rather than being dominated by the 1-2 classes as in §7.2. Validation and test loaders are deliberately left uniform (unweighted) — evaluation must reflect the real, imbalanced population, not the rebalanced training distribution, or the reported metrics would no longer mean "performance on realistic data."
2. **Poisson loss.** The training loss now defaults to `nn.PoissonNLLLoss(log_input=False, full=True)` in place of `nn.MSELoss()`. Since the model's output (`softplus(x) + 1`, §2.4) is already a positive rate rather than a log-rate, `log_input=False` is required; `full=True` adds the Stirling-approximation term so the printed loss is interpretable as an approximate negative log-likelihood, though it does not change the gradient with respect to the prediction.

Both are exposed as CLI flags (`--sampler {uniform,balanced}`, `--loss {mse,poisson}`, both defaulting to the new, fixed behavior), so §7.2's original configuration remains reproducible via `--sampler uniform --loss mse` for comparison.

To isolate the effect of this fix from the effect of more data, it was first re-run on the *same* N=3,000 dataset with the *same* train/val/test split (`--seed 42` unchanged from §7.2) — only `--sampler` / `--loss` changed — saved to `runs_3000_p_wgt/` rather than overwriting the `runs_3000_mse_unw/` baseline. We also added a per-class accuracy breakdown (`per_class_report()` in `train.py`, and the equivalent in `validate.py`) specifically to check whether the rare classes improved, since §7.2

showed aggregate MAE/exact-match/within-±1 accuracy to be a poor lens for that question on their own.

Results. Same 3,000-LC dataset, same 2,100/450/450 split, same seed — only the sampler and loss changed:

Metric	Baseline (`runs_3000_mse_unw`)	Fix (`runs_3000_p_wgt`)
Test MAE	**0.633**	0.749
Exact-match accuracy	**46.2%**	42.7%
Within-±1 accuracy	**92.2%**	86.0%

`n_parent_pulses`	n (test)	Baseline exact-match	Fix exact-match
1	183	35.0%	**51.9%**
2	164	**87.8%**	54.9%
3	73	0.0%	**9.6%**
4	23	0.0%	0.0%
5	6	0.0%	0.0%
6	1	0.0%	0.0%

`runs_3000_p_wgt/compare_3000_balanced_vs_uniform.png` (`compare_runs.py` , §4.3) plots this same per-class comparison as a line graph — baseline peaking sharply at class 2 then dropping to zero beyond it, fix flatter but nonzero through class 3 — making the trade-off visually immediate rather than only readable from the table above.

The two validation figures for the fix (`runs_3000_p_wgt/pulse_count_distribution.png` , `runs_3000_p_wgt/prediction_error_histogram.png`) show the mechanism working exactly as designed: the predicted distribution is no longer collapsed onto 1-2 only — class 3 now receives real predictions (~34 of 73 true class-3 light curves), where the baseline predicted zero. The error histogram is correspondingly *wider* than the baseline's (more ±2 and ±3 errors), which is the direct cause of the drop in within-±1 accuracy.

Interpretation. The fix did exactly what it mechanically should: it stopped the optimizer from taking the "always guess 1 or 2" shortcut, which is visible both in the redistributed predictions and in class 1's accuracy improving substantially (35.0% → 51.9%, since the model is no longer biased toward over-predicting 2 for true-1 examples). But this came at a real cost to class 2 (87.8% → 54.9%) and to the aggregate metrics, and produced only a partial gain on class 3 — with **no** measurable improvement on classes 4-6. That last point is the important one: classes 4, 5, and 6 have only 23, 6, and 1 test examples respectively (and correspondingly few in training — roughly 110, 27, 6 before the split). Inverse-frequency weighting makes the *sampler* present those few examples more often, but with replacement — it cannot manufacture information that isn't there. Training on 25 repeated views of the same ~6 class-5 light curves is a recipe for memorizing those specific 6, not for learning a generalizable "6 parent pulses" pattern; 25 epochs of oversampling a 6-example class is a fundamentally different (and much weaker) regime than 25 epochs over a genuinely larger class-

5 sample. This directly matches the concern raised in §7.3, point 1: **the fix addresses how the existing rare examples are weighted, but not the fact that there are too few of them to learn from in the first place.** It is a real, working answer to "is the model taking a shortcut because of how it's trained" (yes, and this partially corrects it for classes with enough examples to reweight usefully, i.e. class 3), but not a substitute for §7.7's actual test: whether more raw data resolves the classes this fix could not touch.

Given this, the fix is being kept enabled by default for the N=10,000 and N=20,000 runs below (more examples per rare class make oversampling more meaningful, and there is no reason to expect the uniform-sampling/MSE mode-collapse to be *better* at larger N), but §7.7's comparison should be read as "more data, with the fix already applied" rather than an isolated test of data quantity alone — the two effects are intentionally stacked at that point, having already isolated the fix's own effect above.

7.7 Comparative results at N=10,000

The N=10,000 dataset (`runs_10000_p_wgt/`) was generated and trained with the §7.6 fix (balanced sampling + Poisson loss) applied. Its full label distribution:

`n_parent_pulses`	Count	Share
1	4,028	40.3%
2	3,614	36.1%
3	1,700	17.0%
4	511	5.1%
5	125	1.3%
6	15	0.15%
7	6	0.06%
8	1	0.01%

(Aside: these proportions are close to, but flatter than, both the N=3,000 run's *and* a raw, unselected Poisson(`mu0=1.16`) distribution would predict. This is because `generate_GRBs()` 's T90/S-N constraint filter (§2.2) rejects candidate GRBs before they're accepted, and rejection isn't independent of pulse count — a burst with only one short parent pulse is plausibly more likely to fail the T90/S-N thresholds than one with 2-3, skewing the *accepted* population flatter than the *raw* generative distribution. A previously-undocumented detail of the pipeline, noted here for completeness; it doesn't change any conclusion below.)

Results, alongside both 3,000-LC runs:

`n_parent_pulses`	n (test, 3k)	3k baseline	3k + fix	n (test, 10k)	10k + fix
1	183	35.0%	51.9%	595	35.6%
2	164	87.8%	54.9%	549	60.5%

`n_parent_pulses`	n (test, 3k)	3k baseline	3k + fix	n (test, 10k)	10k + fix
3	73	0.0%	9.6%	259	**18.5%**
4	23	0.0%	0.0%	72	1.4%
5	6	0.0%	0.0%	21	0.0%
6	1	0.0%	0.0%	3	0.0%
7	—	—	—	1	0.0%
Aggregate	450	MAE 0.633 / EM 46.2% / ±1 92.2%	MAE 0.749 / EM 42.7% / ±1 86.0%	1,500	MAE 0.750 / EM 39.5% / ±1 87.9%

Both validation figures are in `runs_10000_p_wgt/pulse_count_distribution.png` and `runs_10000_p_wgt/prediction_error_histogram.png`.

Interpretation — a real, if partial, answer to the §7.3 question.

- **Class 3 improved substantially, and roughly in line with the data argument in §6.** With 3.5x more test examples than the 3,000-LC run (73→259), its accuracy roughly doubled (9.6%→18.5%). This is the clearest evidence in this comparison that at least part of the ceiling identified in §7.3 is a genuine data-quantity problem, not solely an information-loss/architecture ceiling — more examples of class 3 measurably helped the model learn it.
- **Classes 5, 6, and 7 still show no improvement whatsoever** — 0% exact-match at every dataset size tested. Even at 10,000 they only reach 21, 3, and 1 test examples respectively (~125, 15, 6 in the full dataset before splitting) — nowhere near the ~250+ that class 3 apparently needed to show real improvement. Consistent with the earlier estimate that these classes would need far more data (potentially 100k+) before scale alone resolves them.
- **Class 4 barely moved** (0.0%→1.4%) despite tripling in test-set size (23→72) — sitting between class 3's clear improvement and classes 5-7's total stagnation. This suggests 72 examples is still below whatever threshold mattered for class 3, rather than close to it.
- **An unexpected result: overall exact-match accuracy went down again** (42.7%→39.5%), and class 1's accuracy reverted most of the way back toward the un-fixed baseline (51.9%→35.6%, vs. baseline's 35.0%), even though class 2 improved (54.9%→60.5%). The most honest explanation is the one already flagged as a limitation (§8): this is a single training run per dataset size, with no repeated-seed averaging, so some of this movement is plausibly just run-to-run variance rather than a systematic effect of dataset size. A second, more speculative but plausible contributing factor: `WeightedRandomSampler`'s rebalancing is relative to each dataset's *own* class proportions, which are not identical between the 3,000 and 10,000 runs (see the distribution tables above) — so the "effective" balanced training mix the model actually trains on isn't the same shape at both sizes, which could shift the decision boundary between classes 1 and 2 in a way that isn't simply "more data, monotonically better." Either way, this is a genuine reason to trust the per-class breakdown over aggregate exact-match accuracy when judging whether scaling data is helping — the aggregate number moved in a direction that would look like "getting worse" while the class this experiment was actually targeting (3) demonstrably improved.

Bottom line for the question this comparison was designed to answer: the evidence is genuinely mixed, not a single clean verdict. Part of the tail (class 3, plausibly class 4 with more data) responds to more data — supporting the imbalance/data-quantity explanation. Part of it (classes 5-7) shows no response across a 3.3x increase in N — consistent with those specific classes needing substantially more data than tested here, rather than necessarily an information-loss ceiling; nothing here rules out data quantity as the eventual fix for classes 5-7, only that 10,000 isn't enough yet. Whether N=20,000 is worth running to test that further is an open decision (§7.1) — that run is currently on hold.

7.8 Discussion: what the two comparisons tell us together, and what needs to change

Run in isolation, neither §7.6 nor §7.7 would be conclusive. Read together, they let us attribute *which* rare classes are failing for *which* reason, rather than treating "the model is bad at rare classes" as one undifferentiated problem.

What the two comparisons tell us:

- **Class 3 is a data-quantity problem, and it is solved by scale.** 73→259 test examples (3k→10k) took its accuracy from 9.6%→18.5% — roughly doubling in proportion to roughly doubling the data. This is the clean, unambiguous result of this whole exercise: more data works, when there is a realistic amount of it to add.
- **Classes 5, 6, and 7 are not solved by a 3.3x increase in data** — flat 0% at both dataset sizes, because even 10,000 total light curves yields only 21, 3, and 1 test examples for them respectively. That is not evidence that "scale doesn't work" for these classes; it is evidence that 3.3x more of an already-tiny number is still a tiny number. Given the fixed Poisson rate, bringing class 7 to where class 3 already sits (~250 examples) would require on the order of 250/0.0006 ≈ 400,000+ light curves at this `mu0` — scale alone is not a practical fix for these classes at the rates tested here.
- **Class 4 is genuinely ambiguous** — 0.0%→1.4% despite tripling its example count (23→72). It neither clearly responds the way class 3 did nor stays as flatly dead as classes 5-7. This is the class an N=20,000 run would be most informative about.
- **The §7.6 fix is necessary but not sufficient, and its effect on the dominant classes is uneven across dataset sizes.** It is what let class 3 improve at all — the unfixed baseline predicted nothing above 2, ever — but class 1's accuracy moved non-monotonically across the three runs (35.0% → 51.9% → 35.6%, baseline → 3k-fix → 10k-fix), which is a sign the reweighting mechanism itself behaves somewhat differently at different dataset sizes, not simply "more data, uniformly better."
- **Aggregate metrics (MAE, exact-match accuracy) are actively misleading here.** Both got slightly worse from the 3k-fix run to the 10k-fix run, even though the class this whole experiment was targeting (3) clearly improved. Judging "did this help" from the aggregate number alone would give the wrong answer — the per-class breakdown is not optional context, it is the load-bearing evidence.

What needs to change to improve accuracy, in priority order (based on what the data actually showed, not what seemed promising before testing):

1. **Targeted data generation for classes 5-7, not just more of the same distribution.** Since `mu0=1.16` is fixed, every additional light curve is disproportionately a 1 or 2. Generating separate batches at a deliberately higher `mu0` — to naturally produce far more 5/6/7-pulse bursts — and mixing them into training is the direct fix; waiting for uniform-distribution scale to do it "naturally" is not practical at the rates involved (previous point).
2. **Repeat runs across multiple seeds before trusting any single comparison further.** The class-1 non-monotonicity above is exactly the kind of thing that could be run-to-run noise rather than a real effect of dataset size — every result so far is a single seed, single split (§8), so we currently cannot tell which it is.
3. **Reconsider the *strength* of the reweighting, not just whether to use it.** Full inverse-class-frequency weighting may be over-correcting for the dominant classes — class 2's accuracy dropped from 87.8% (baseline) to 54.9% (3k-fix) and only partially recovered at 10k (60.5%), never approaching its baseline level. A softer scheme, e.g. weighting by `1/sqrt(count)` instead of `1/count`, would rebalance rare classes less aggressively and might reduce this collateral damage while still helping class 3.
4. **Architecture changes become worth trying specifically once class 4's ambiguous result is resolved.** If class 4 does not clearly respond to a future data increase either, that would be evidence that global average pooling (which discards positional/count information, §2.4) is capping what is learnable regardless of data or reweighting — at that point, a representation closer to "count distinct activations" rather than "average activation" is the more promising lever, not more data.
5. **N=20,000 (or targeted generation instead of it, per point 1)** is the direct next test for whether class 4 keeps responding to scale the way class 3 did, or plateaus like classes 5-7 — currently on hold pending a decision on which of these is the better use of that time.

7.9 Visual comparisons: balanced vs. uniform sampling, at both dataset sizes

§7.6 and §7.7 presented the balanced-vs-uniform comparison as tables. `compare_runs.py` (§4.3) renders the same per-class accuracy comparison as a line graph instead, at both dataset sizes now that a matching uniform/MSE run exists at N=10,000 (`runs_10000_mse_unw/`, trained identically to `runs_10000_p_wgt/` — same data, same split — except `--sampler uniform --loss mse`):

- `runs_3000_p_wgt/compare_3000_balanced_vs_uniform.png` — N=3,000.
- `runs_10000_p_wgt/compare_10000_balanced_vs_uniform.png` — N=10,000.

The N=10,000 comparison surfaces something the table form didn't make as visible: **the uniform/MSE baseline itself is no longer stuck at exactly zero on class 3 the way it was at N=3,000** — it reaches 3.9% purely from having more data, before any fix is applied. The fix still does substantially better on class 3 (18.5%), but this is a reminder that "more data" and "the fix" are not fully independent axes — more data helps a little even without the fix, on top of what the fix adds.

Does the fix's benefit change with dataset size? `compare_fix_effect_by_size.py` (new) directly plots this: for each class, `(balanced accuracy) - (uniform accuracy)` at N=3,000 vs. at N=10,000, saved to `runs_10000_p_wgt/compare_fix_effect_3000_vs_10000.png`.

`n_parent_pulses`	Fix effect at N=3,000	Fix effect at N=10,000
1	**+16.9%**	−3.5%
2	−32.9%	**−14.8%**
3	+9.6%	**+14.7%**
4	0.0%	+1.4%
5-7	0.0%	0.0%

Three distinct patterns, not one uniform trend:

- **Class 3 (the class this fix was actually meant to help): the benefit is stable and, if anything, grows with more data** (+9.6% → +14.7%) — the strongest evidence in this whole comparison that the fix is doing what it was designed to do, and that it doesn't fade as N grows.
- **Class 2 (the dominant class the fix costs accuracy on): the cost shrinks substantially with more data** (−32.9% → −14.8%) — suggesting that at larger N, the fix's collateral damage to the dominant classes may continue to shrink even as its benefit to class 3 holds.
- **Class 1: the effect reverses sign** (+16.9% → −3.5%) — positive at 3,000, slightly negative at 10,000. Per §7.7's discussion of the same reversal, this is exactly the kind of movement that could be genuine sampler/dataset-size interaction, or could be single-seed noise (§8) — it cannot be distinguished from this comparison alone, and is flagged rather than over-interpreted.

Taken together, this is a more encouraging picture for the fix at scale than the raw aggregate-accuracy numbers in §7.7 suggested on their own: the class it targets keeps benefiting, and its cost to the dominant classes appears to be shrinking, not growing, as N increases.

7.10 Isolating the fix's two components: sampler vs. loss, plus a third strategy

The §7.6 fix bundles two changes together (balanced sampling + Poisson loss). To find out how much each contributes independently, two more configurations were trained on the identical N=3,000 dataset/split as §7.2/§7.6, using the new `--preset` flag (§4.3):

- `poisson-only` (uniform sampler + Poisson loss): isolates the loss change alone.
- `capped` (uniform sampler + MSE loss, but the *training set itself* undersampled to at most 1,000 examples per class before splitting, via the new `cap_per_class()` in `dataset.py`, §4.3): a third, different strategy for imbalance — shrinking the data's imbalance directly, rather than reweighting how it's sampled or changing the loss.

Results at N=3,000 (`runs_3000_p_wgt/compare_3000_all_four.png`):

n_parent_pulses	Baseline	Poisson-only	Capped (1000/class)	Fix (balanced+Poisson)
1	35.0%	35.5%	**1.3%**	51.9%
2	87.8%	81.7%	**100.0%**	54.9%
3	0.0%	1.4%	0.0%	**9.6%**
4-6	0.0%	0.0%	0.0%	0.0%

Two clear findings:

1. **The loss change alone barely moves anything.** `poisson-only`'s class 3 accuracy (1.4%) is essentially unchanged from baseline (0.0%) — noise at this sample size, not a real effect — and classes 1/2 are statistically indistinguishable from baseline too. Nearly all of the §7.6 fix's actual benefit on class 3 (0.0%→9.6%) comes from the **balanced sampler**, not the Poisson loss. The loss change may still be the theoretically more appropriate choice for count data (§2.5), but on its own it is not what corrects the mode-collapse behavior.
2. **Undersampling to a cap doesn't fix mode collapse — it just relocates it.** Capping classes 1 and 2 to 1,000 examples each (now nearly identical counts) didn't flatten the model's behavior; it collapsed onto class 2 *instead of* class 1 (100.0% accuracy on class 2, 1.3% on class 1 — consistent with the model simply always guessing "2" rather than genuinely discriminating between them). This is a real, informative negative result: reducing the dominant classes' raw example count, without also changing the loss or reweighting the sampler, does not address *why* MSE regression defaults to a single dominant guess (§7.3, point 2) — it only changes *which* value that default lands on, apparently based on whichever of the now near-tied classes 1/2 the optimizer happened to settle toward first.

Confirmed at N=10,000 too, for the loss-alone condition: `runs_10000_p_unw/` (uniform sampler + Poisson loss, full 10,000-LC dataset) reaches 2.7% on class 3 — statistically indistinguishable from the 10,000-LC baseline's 3.9% (§7.9), and nowhere near the fix's 18.5%. Same conclusion as at N=3,000: the loss change alone isn't what's doing the work.

But the capped condition behaves completely differently at N=10,000 — and genuinely well. `runs_10000_mse_cap/` (`compare_10000_all_four.png`):

n_parent_pulses	Baseline	Poisson-only	Capped (1000/class)	Fix (balanced+Poisson)
1	39.2%	42.9%	**2.7%**	35.6%
2	75.2%	75.2%	55.7%	60.5%
3	3.9%	2.7%	**60.8%**	18.5%
4	0.0%	0.0%	0.0%	1.4%
5-7	0.0%	0.0%	0.0%	0.0%

At N=3,000, capping only brought two classes (1 and 2) down to an equal size (1,000 each), and the model simply collapsed onto class 2 instead of class 1 — no real discrimination gained. At N=10,000, capping brings **three** classes to an equal size — class 3 (1,700 raw examples) is now also capped to 1,000, tied with classes 1 and 2 — and something different happens: the model reaches **60.8% on**

class 3, dramatically better than any other condition tested (next best: the fix's 18.5%), while nearly abandoning class 1 (2.7%). This looks like genuine discrimination between classes 2 and 3 (both around 55-61%), not another single-class collapse — the model found a real signal separating them once they had truly equal representation, rather than merely a majority/minority relationship.

This is the strongest evidence in this whole comparison that **the number of classes tied at parity matters, not just "some undersampling happened."** Two tied classes (N=3,000) reproduced the same failure mode under a different label; three tied classes (N=10,000) broke out of it convincingly for the class in the middle of that trio. It also comes with the same trade-off in a sharper form: class 1 was sacrificed almost entirely to get class 3's gain, and classes 4+ (never brought to parity, still only 511/125/15/6/1 raw examples) saw no benefit at all. Whether this generalizes — e.g. would capping at a size that also equalizes class 4 unlock it too, at the cost of even more of classes 1-3 — is a direct, concrete next experiment this result motivates, not yet run.

7.11 The full loss × weighting comparison, and an unexpected result: MSE beats Poisson under weighted sampling

§7.10 only tested `poisson-only` and `capped`; the missing cell in the 2×4 factorial (§7, overview table) was **Poisson loss combined with capping** (`p_cap`), plus a full six-way comparison including `mse_wgt`. Both are now complete at N=3,000 (`runs_3000_p_wgt/compare_3000_six_conditions.png`):

n_parent_pulses	mse_unw	mse_wgt	mse_cap	p_unw	p_wgt	p_cap
1	35.0%	4.4%	1.3%	35.5%	51.9%	13.3%
2	87.8%	79.3%	100.0%	81.7%	54.9%	96.0%
3	0.0%	**21.9%**	0.0%	1.4%	9.6%	0.0%
4	0.0%	**4.3%**	0.0%	0.0%	0.0%	0.0%
5-6	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%

Two results stand out:

1. **`p_cap` confirms §7.10's capping conclusion is about the sampler choice, not the loss.** `p_cap` (13.3% / 96.0% / 0.0%) shows the same collapse-onto-class-2 pattern as `mse_cap` (1.3% / 100.0% / 0.0%) — capping relocates the collapse regardless of which loss function rides on top. The mechanism identified in §7.10 (two classes tied at equal size, MSE-style mean-seeking behavior under *either* loss when the sampler itself isn't doing anything to counter it) holds for Poisson loss too.
2. **`mse_wgt` outperforms `p_wgt` on the classes the fix was meant to help** — 21.9% vs. 9.6% on class 3, 4.3% vs. 0.0% on class 4 — which is the opposite of what §2.5's theoretical argument for Poisson loss (matching the count-data nature of the label) would predict. This was confirmed again at N=10,000: `runs_10000_mse_wgt` reaches **28.6%** on class 3 (vs. `runs_10000_p_wgt`'s 18.5%) and 1.4% on class 4 — both sizes agree. The balanced *sampler* appears to be doing more of the real work than previously credited to it in §7.10, and pairing it with MSE rather than Poisson currently gives the best rare-class results of any configuration tested so far (at the cost of class 1, which drops further under `mse_wgt` than under `p_wgt` —

4.4%/19.5% vs. 51.9%/35.6% at N=3,000/10,000 respectively). Why Poisson loss underperforms its theoretical motivation here has not been investigated — a plausible direction is that `PoissonNLLLoss`'s asymmetric penalty (harsher on under- than over-prediction at these small count values) interacts poorly with `WeightedRandomSampler`'s already-aggressive rebalancing, but this is speculation, not demonstrated.

The N=10,000 six-condition chart (`runs_10000_p_wgt/compare_10000_six_conditions.png`) confirms `p_cap` follows the same "three classes tied at parity → real discrimination" pattern as `mse_cap` (§7.10), but strikes a different balance among the tied trio: `p_cap` reaches 26.4%/61.4%/42.0% on classes 1/2/3 versus `mse_cap`'s 2.7%/55.7%/60.8% — `p_cap` recovers much more of class 1 (26.4% vs. 2.7%) at the cost of some of class 3 (42.0% vs. 60.8%). Both are dramatically better than any `unw` / `wgt` condition on class 3, reinforcing §7.10's core finding (parity among tied classes matters more than which loss sits on top), while showing the loss function does still shift *where* the resulting trade-off lands.

The `uni` (uniform-by-construction dataset) conditions, N=3,000. Unlike `unw` / `wgt` / `cap`, this dataset is not a resampling of the natural Poisson-imbalanced data at all — it is 300 separately-harvested light curves per bin (§7 overview), so the sampler has nothing left to correct and `p_uni` / `mse_uni` use the plain/uniform sampler on top of it. Bins 9-10 were dropped from the harvesting range for cost reasons (§9) — this run covers bins 1-8 only, 2,400 light curves total, 1,680/360/360 train/val/test:

`n_parent_pulses`	`p_uni`	`mse_uni`
1	0.0%	0.0%
2	2.2%	0.0%
3	22.2%	26.7%
4	37.8%	**57.8%**
5	**44.4%**	17.8%
6	13.3%	6.7%
7	0.0%	2.2%
8	0.0%	0.0%

Run	Test MAE	Exact-match	Within-±1
`runs_3000_p_uni`	1.731	**15.0%**	**45.8%**
`runs_3000_mse_uni`	1.861	13.9%	40.6%

This is the best per-class spread of any condition tested at N=3,000 — every bin except the extremes (1, 8) gets meaningfully engaged, and the aggregate within-±1 accuracy (40.6-45.8%) beats every `wgt` / `cap` condition in §7.10/§7.11 by a wide margin. It comes at the cost of class 1 collapsing to 0%, but that mirrors the tied-classes mechanism from §7.10 — 1 and 2 are no longer the overwhelming majority here, so the model has no incentive to default to them, and instead spreads its guesses across the bins that actually have equal representation. This is the cleanest evidence in the whole

investigation that genuine data balance (not just reweighting a still-imbalanced dataset) is the more effective remedy, at the cost of requiring far more generation effort per example (§9).

The `uni` conditions at N=10,000 (bins 1-7, 1,000/bin, 7,000 light curves total, 1,050/1,050/... test split):

`n_parent_pulses`	`mse_uni`	`p_uni`
1	10.7%	7.3%
2	28.0%	22.0%
3	18.0%	16.0%
4	26.7%	**46.7%**
5	**48.0%**	41.3%
6	10.7%	5.3%
7	0.0%	0.0%

Run	Test MAE	Exact-match	Within-±1
`runs_10000_mse_uni`	1.475	**20.3%**	54.5%
`runs_10000_p_uni`	**1.438**	19.8%	**55.6%**

(`mse_uni` edges `p_uni` on exact-match, 20.3% vs. 19.8% — the only metric where MSE wins here; `p_uni` wins MAE and within-±1.) Two things change from the N=3,000 result (§7.11): first, **the extra data closes most of the gap between the two losses** — at N=3,000 `p_uni` beat `mse_uni` clearly on every metric (MAE 1.731 vs. 1.861, within-±1 45.8% vs. 40.6%); at N=10,000 the two are close enough (1.438 vs. 1.475 MAE) that which loss "wins" depends on which metric is read, consistent with §6's general prediction that more data should shrink the gap between reasonable modeling choices. Second, **every metric improved substantially with more data** — within-±1 accuracy rose from 40.6-45.8% at N=3,000 to 54.5-55.6% at N=10,000, the largest size-driven improvement of any regime in this report, which makes sense: this is the one dataset where every class (not just the majority) actually gets more absolute examples to learn from as N grows, since the classes were equal-sized by construction to begin with. Per-class comparison chart: `compare_uni_10000.png`.

7.12 Master comparison table (all conditions, both dataset sizes)

Every run's aggregate metrics in one place — the per-class breakdowns behind these numbers are in §7.2, §7.6, §7.7, §7.10, and §7.11; this table is for the top-line comparison only. Every run's own `pulse_count_distribution.png` and `prediction_error_histogram.png` (§2.7) exist in its run directory, generated via `validate.py`. **The `uni` rows are not directly comparable to the rest of this table** — they are evaluated on a different label range (bins 1-8 only, vs. the natural dataset's long tail) and a smaller/differently-composed test split, so their higher MAE reflects a harder, wider-spread label distribution rather than worse performance; see §7.11's discussion for the fair, per-class comparison.

Run	N	Loss	Weighting	Test MAE	Exact-match	Within-±1
`runs_3000_mse_unw`	3,000	MSE	unweighted	0.633	**46.2%**	92.2%
`runs_3000_mse_wgt`	3,000	MSE	weighted	0.758	34.4%	91.3%
`runs_3000_mse_cap`	3,000	MSE	capped	0.720	37.6%	92.3%
`runs_3000_p_unw`	3,000	Poisson	unweighted	0.658	44.4%	91.6%
`runs_3000_p_wgt`	3,000	Poisson	weighted	0.749	42.7%	86.0%
`runs_3000_p_cap`	3,000	Poisson	capped	0.691	40.6%	**92.3%**
`runs_3000_mse_uni`	3,000 (2,400 uniform-by-construction, bins 1-8)	MSE	uniform (`uni`)	1.861	13.9%	40.6%
`runs_3000_p_uni`	3,000 (2,400 uniform-by-construction, bins 1-8)	Poisson	uniform (`uni`)	1.731	15.0%	45.8%
`runs_10000_mse_unw`	10,000	MSE	unweighted	0.670	43.7%	91.6%
`runs_10000_mse_wgt`	10,000	MSE	weighted	0.801	35.9%	86.5%
`runs_10000_mse_cap`	10,000	MSE	capped	0.860	32.6%	83.2%
`runs_10000_p_unw`	10,000	Poisson	unweighted	0.660	**45.0%**	91.8%
`runs_10000_p_wgt`	10,000	Poisson	weighted	0.750	39.5%	87.9%
`runs_10000_p_cap`	10,000	Poisson	capped	0.843	35.7%	83.1%
`runs_10000_mse_uni`	pending	MSE	uniform (`uni`)	—	—	—
`runs_10000_p_uni`	pending	Poisson	uniform (`uni`)	—	—	—

Read this table with §7.2's caveat in mind: the unweighted rows' aggregate numbers look best, and that is exactly the mode-collapse artifact this whole investigation is about, not evidence those runs are actually "better." `mse_unw` and `p_unw` top exact-match accuracy because guessing 1 or 2 is right most of the time when 77-86% of the test set genuinely is 1-2 pulses (§7.2) — the aggregate number rewards that shortcut. The weighted/capped rows score *lower* in this table precisely because they sacrifice some of that easy accuracy to actually engage with classes 3+, which is the entire point (§7.6, §7.10, §7.11). Judge these runs by the per-class tables and comparison charts in §7.6-§7.11, not by sorting this table on exact-match accuracy.

8. Limitations

- `n_sig_pulses` remains broken upstream (§3.2) and unused; only `n_parent_pulses` is a reliable label from this pipeline at present.
- The model is trained on a single, fixed instrument/parameter regime (Swift v36-calibrated). It has no basis for generalizing to BATSE light curves, or to a different SS96 parameter set,

without retraining on data generated under those settings.
- Each reported run reflects a single train/val/test split and a single random seed; metrics are not averaged over repeated splits or seeds.
- **The model cannot be directly verified against real GRB light curves.** `n_parent_pulses` is a latent quantity — it exists only because the synthetic pipeline knows exactly how many pulses it drew from the SS96 avalanche model when generating a light curve. No real catalog (Swift/ BAT, BATSE) records a "true number of parent pulses" for an observed burst; it is not a measured physical quantity, it is an artifact of this specific generative model, so there is no ground-truth label to check a real-light-curve prediction against. The closest available proxy is independent peak-counting: `genetic_algorithm/geneticgrbs.py` already references precomputed MEPSA significant-peak counts for real Swift/BAT bursts (`mepsa_out_file_list`) and `statistical_test.py` has a matching `n_sig_pulses` field (currently broken upstream, above) — comparing the CNN's prediction on a real light curve to MEPSA's peak count would be a consistency check between two different pulse-counting methods, not ground-truth verification, since MEPSA counts only peaks *above a detection threshold* while the CNN was trained to recover the underlying generative count (including overlapping/sub-threshold pulses a peak-finder would miss) — agreement wouldn't prove correctness and disagreement wouldn't prove error. This is moot for now regardless: both the real catalog and the precomputed MEPSA files are hardcoded to remote server paths (`/astrodata/guidorzi/...` , `/beegfs/mbulla/...` , §4.1) that don't exist on this machine, so pursuing this would first require acquiring that data. `lc_gallery.py` (§10.3) offers a much weaker, purely visual substitute: representative synthetic light curves by pulse count, for eyeballing rough morphological similarity against a real burst, which is not a validation method.

9. A second parameter regime: `mu0=10`

Everything in §7 uses the Bazzanini-calibrated Swift/BAT value `mu0=1.16` (config.ini), which sets the Poisson-distributed number of parent pulses per light curve to a mode of 1 and a heavy right tail. `mu0` is the *only* SS96 parameter that controls `n_parent_pulses` (§2.1, §7 overview) — the other six (`mu, alpha, delta1, delta2, tau_min, tau_max`) shape pulse *shape and timing*, not count. To see how the whole investigation's findings change under a distribution centered somewhere else, `mu0` was raised to 10 (config.ini, old value commented out rather than deleted) and a fresh 3,000- and 10,000-LC dataset generated under it, then run through the same training conditions as §7.

Scope reduction (later completed). Generation and training at N=10,000 under `mu0=10` proved far more expensive than anticipated (see cost diagnostic below), and this track was initially run against a hard deadline — only the Poisson-loss conditions (`unw` , `wgt` , `cap`) were trained at N=10,000 at first, with the three MSE conditions dropped for that size. Once time allowed, all three MSE conditions (`mse_unw` , `mse_wgt` , `mse_cap`) were completed too, so the full 2×3 factorial (both losses × three weighting strategies) is now done at **both** dataset sizes — §9.2 reports all 6 conditions at N=10,000.

Cost diagnostic. A 100-LC timing probe at `mu0=9` and `mu0=10` (`n_cut=2500`, same filters as production) found generation costs ~2.5s per *accepted* GRB at these values, versus ~0.4-0.9s at the natural `mu0=1.16` — and only a ~9% exact-match rate to the target bin (vs. the naive Poisson-mode estimate of ~12.5%), because `n_cut=2500` — a shared pulse-count budget across a whole light curve's avalanche — is hit far more often when `mu0` itself is 9-10, discarding many otherwise-plausible candidates. This cost is specific to the `uni` (uniform-by-construction) harvesting method, which sweeps `mu0` per target bin (§7.11) — it is why `generate_uniform_bins.py` gained a `--max-bin` argument, and why both `uniform_3000` / `uniform_10000` were capped below the originally planned 1-10 range. The same growth curve kept biting at the smaller cap too: `uniform_3000` finished bins 1-8 without issue, but `uniform_10000` (needing more than 3x as many matches per bin) hit a wall at bin 7 — its harvest rate dropped to roughly a quarter of bins 1-6's pace — so bin 8 was dropped for that dataset specifically, leaving `uniform_10000` capped at bins 1-7 rather than 1-8 (§7 overview table, §10).

9.1 Results at N=3,000

n_parent_pulses	mse_unw	mse_wgt	mse_cap	p_unw	p_wgt	p_cap
2	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
3	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
4	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
5	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
6	0.0%	3.4%	0.0%	0.0%	**17.2%**	0.0%
7	0.0%	10.3%	0.0%	0.0%	12.8%	0.0%
8	10.7%	10.7%	10.7%	10.7%	12.5%	10.7%
9	**30.2%**	14.0%	30.2%	30.2%	14.0%	30.2%
10	**34.4%**	10.9%	34.4%	34.4%	12.5%	34.4%
11	17.8%	8.9%	17.8%	20.0%	6.7%	20.0%
12	0.0%	15.0%	0.0%	10.0%	**17.5%**	10.0%
13	0.0%	**18.4%**	0.0%	0.0%	10.5%	0.0%
14	0.0%	12.0%	0.0%	0.0%	4.0%	0.0%
15+	0.0%	≤9.1%	0.0%	0.0%	≤9.1%	0.0%

Run	Test MAE	Exact-match	Within-±1
runs_3000mu10_mse_unw	2.429	10.9%	34.2%
runs_3000mu10_mse_wgt	2.802	10.0%	33.1%
runs_3000mu10_mse_cap	2.429	10.9%	34.2%
runs_3000mu10_p_unw	2.427	12.0%	36.4%
runs_3000mu10_p_wgt	2.793	10.4%	32.4%
runs_3000mu10_p_cap	2.427	12.0%	36.4%

Full comparison chart: `compare_mu10_3000_all.png` (all 6 conditions).

Two findings, both consistent with §7's mechanism:

1. **The same mode-collapse pattern reappears, just recentered.** With `mu0=10`, the label distribution's mode sits at 9-10 instead of 1-2 (2/3/4/5/8/11-pulse counts are all thin — see the label-distribution printout in any `unw` run's log), and `mse_unw` / `p_unw` collapse onto *that* mode (30-34% on classes 9-10, 0% on everything below 6) exactly as `mse_unw` / `p_unw` collapsed onto classes 1-2 at `mu0=1.16` (§7.2). The weighted conditions again spread accuracy across more classes (nonzero on 6, 7, 12, 13, 14) at the cost of the dominant bins — the same trade-off as §7.6/§7.10, now demonstrated under a second, independently-generated distribution.
2. **Both `p_cap` and `mse_cap` are numerically identical to their unweighted counterparts here — a no-op, not a fix, regardless of loss function.** `--cap-per-class 1000` only removes examples from classes *above* the cap; at `mu0=10` /N=3,000 the largest class (10 pulses) has only 374 examples, so nothing was ever removed. `p_cap` is byte-for-byte identical to `p_unw`, and `mse_cap` is byte-for-byte identical to `mse_unw` (same per-class numbers, same aggregate MAE 0.429/exact 10.9%/within-±1 34.2%, confirmed once `mse_cap` finished training) — the loss function was never the reason capping did nothing here, the dataset simply never triggered it. This is not a new finding about capping's effectiveness — §7.10/§7.11 already showed capping only helps once enough classes are tied at parity — it's a reminder that the cap threshold has to be chosen relative to the actual class sizes in play, which shrink at N=3,000 once the label distribution is this spread out (23 distinct values, §7.9's table pattern repeated). §9.2 shows the same cap becomes active, non-trivially, once N grows to 10,000 and several classes naturally exceed 1,000.

9.2 Results at N=10,000

All 6 conditions (MSE conditions completed after the initial scope reduction, §9's note above):

n_parent_pulses	mse_unw	mse_wgt	mse_cap	p_unw	p_wgt	p_cap
2-3	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
4	0.0%	12.9%	0.0%	0.0%	0.0%	0.0%
5	0.0%	7.5%	0.0%	0.0%	1.9%	0.0%
6	1.0%	9.0%	1.0%	0.0%	9.0%	0.0%
7	7.5%	7.5%	8.3%	1.7%	8.3%	0.6%
8	12.4%	10.6%	12.7%	10.6%	12.9%	7.5%
9	16.5%	16.0%	21.4%	24.5%	14.9%	22.7%
10	**39.2%**	14.4%	**46.6%**	35.1%	17.5%	32.5%
11	28.3%	14.5%	25.2%	25.9%	20.5%	**33.1%**
12	3.4%	**15.9%**	0.0%	11.0%	**15.9%**	13.7%
13	0.0%	12.1%	0.0%	1.9%	11.2%	3.9%

n_parent_pulses	mse_unw	mse_wgt	mse_cap	p_unw	p_wgt	p_cap
14	0.0%	12.2%	0.0%	1.2%	8.5%	0.0%
15	0.0%	3.3%	0.0%	0.0%	4.9%	0.0%
16	0.0%	0.0%	0.0%	0.0%	2.8%	0.0%

Run	Test MAE	Exact-match	Within-±1
runs_10000mu10_mse_unw	2.291	12.7%	39.3%
runs_10000mu10_mse_wgt	2.715	11.6%	33.9%
runs_10000mu10_mse_cap	2.391	12.4%	37.6%
runs_10000mu10_p_unw	2.297	13.1%	39.9%
runs_10000mu10_p_wgt	2.631	12.3%	35.3%
runs_10000mu10_p_cap	2.444	11.9%	37.0%

Full comparison chart: `compare_mu10_10000_all.png` (all 6 conditions).

`mse_wgt` reaches the sparse tails about as well as `p_wgt` does, extending the §7.11 finding to this regime. `mse_wgt` is nonzero on 4, 5, 13, 14, 15 — a near-identical reach to `p_wgt`'s (5, 6, 13, 14, 15, 16) — and `mse_unw` / `mse_cap` collapse onto the 9-11 mode exactly like `p_unw` / `p_cap` do (`mse_cap` even peaks higher at bin 10, 46.6%, than any other condition — the sharpest collapse of the six). `p_wgt` still has the better aggregate within-±1 (35.3% vs. `mse_wgt`'s 33.9%), so it remains the pick for this dataset (§10.4), but the margin is much narrower than §7.6-§7.11's natural-data comparisons — both weighted conditions are now doing genuinely similar work, just with `p_wgt` slightly ahead.

The N=3,000→10,000 jump changes `p_cap` from a no-op into something real, but a limited kind of real. At N=3,000 (§9.1), the largest class had only 374 examples, well under the 1,000 cap, so `p_cap` was byte-for-byte identical to `p_unw`. At N=10,000, classes 8-11 each exceeded 1,000 naturally and were genuinely undersampled down to 1,000 (9,227 of 10,000 light curves kept overall). But `p_cap`'s per-class numbers above are still close to `p_unw`'s at the sparse ends — 0% on classes 2-6 either way — because classes 8-11 were already comparably sized *before* capping (873-1,000+ each); tying them at exact parity doesn't touch the real structural gap between that dense core and the genuinely rare tails (1-6, 15+), which the cap never reaches since it only removes examples from classes *above* the threshold. This is the same lesson as §7.10's "how many classes end up tied at parity" finding, sharpened: parity among already-populous classes doesn't rescue classes that were never populous to begin with.

The balanced sampler (`wgt`, either loss) is the only strategy that reaches the sparse tails at all — nonzero accuracy on 4-6 and 12-16, none of which any `unw` / `cap` condition (either loss) touches — at the cost of the dominant bins (10: 14.4-17.5% vs. 32.5-46.6% for the rest). This mirrors §7.6/§9.1's core trade-off exactly, now confirmed at a third dataset size/regime combination and across both losses: reweighting how existing rare examples are *seen* during training reaches further into the tails than any resampling of the existing data, capped or not, can.

10. Master comparison across every regime, N=3,000

§7.12 compared every condition within the natural (`mu0=1.16`) factorial; §9 and §7.11 each reported their own regime's conditions separately. This section puts all 14 conditions trained at N=3,000 — 6 natural, 6 at `mu0=10` , 2 `uni` — on one chart and one table (`runs_3000_p_wgt/compare_all_3000.png` , `runs_3000_p_wgt/compare_all_3000_table.md`):

Regime	Condition	Test MAE	Exact-match	Within-±1
natural (mu0=1.16)	`mse_unw`	0.633	46.2%	92.2%
natural (mu0=1.16)	`mse_wgt`	0.758	34.4%	91.3%
natural (mu0=1.16)	`mse_cap`	0.720	37.6%	92.3%
natural (mu0=1.16)	`p_unw`	0.658	44.4%	91.6%
natural (mu0=1.16)	`p_wgt`	0.749	42.7%	86.0%
natural (mu0=1.16)	`p_cap`	0.691	40.6%	92.3%
mu0=10	`mse_unw`	2.429	10.9%	34.2%
mu0=10	`mse_wgt`	2.802	10.0%	33.1%
mu0=10	`mse_cap`	2.429	10.9%	34.2%
mu0=10	`p_unw`	2.427	12.0%	36.4%
mu0=10	`p_wgt`	2.793	10.4%	32.4%
mu0=10	`p_cap`	2.427	12.0%	36.4%
uniform-by-construction	`mse_uni`	1.861	13.9%	40.6%
uniform-by-construction	`p_uni`	1.731	15.0%	45.8%

Read the chart's bar heights within a regime, not across regimes. Within-±1 accuracy depends on how wide the true-label distribution is, not just on model quality — natural's labels sit almost entirely on 1-2, so "within one pulse" is a low bar cleared 86-92% of the time regardless of which sampler/loss is used; `mu0=10` 's labels spread over roughly 2-25, so the identical ±1 tolerance is a much stricter test, and 32-36% there is not directly weighable against 86-92% at natural. The uniform regime sits in between (labels confined to 1-8, but genuinely flat rather than concentrated). Comparing bar heights *within* a color group is where the real signal is — e.g. `p_unw` / `p_cap` beating `p_wgt` inside the `mu0=10` group (36.4% vs. 32.4%) rearranges §7's ordering *because the class sizes that made* `wgt` *worth the trade-off in §7.6 don't exist at* `mu0=10` *and N=3,000* (§9's point 2 — nothing was imbalanced enough for `wgt` 's reweighting to have much to fix). Across regimes, the more meaningful comparison is qualitative: which regime makes rare classes reachable at all (§7.11, §9) — a question the per-class tables in §7.6-§7.11 and §9.1 answer, not this aggregate chart.

10.1 Master comparison across every regime, N=10,000

All 14 conditions complete at N=10,000 — the full natural (`mu0=1.16`) factorial, the full `mu0=10` factorial (both losses, all three weighting strategies — completed in full once time allowed, §9), and

both `uni` conditions — on one chart and table (`runs_10000_p_wgt/compare_all_10000.png` , `runs_10000_p_wgt/compare_all_10000_table.md`):

Regime	Condition	Test MAE	Exact-match	Within-±1
natural (mu0=1.16)	`mse_unw`	0.670	43.7%	91.6%
natural (mu0=1.16)	`mse_wgt`	0.801	35.9%	86.5%
natural (mu0=1.16)	`mse_cap`	0.860	32.6%	83.2%
natural (mu0=1.16)	`p_unw`	0.660	45.0%	91.8%
natural (mu0=1.16)	`p_wgt`	0.750	39.5%	87.9%
natural (mu0=1.16)	`p_cap`	0.843	35.7%	83.1%
mu0=10	`mse_unw`	2.291	12.7%	39.3%
mu0=10	`mse_wgt`	2.715	11.6%	33.9%
mu0=10	`mse_cap`	2.391	12.4%	37.6%
mu0=10	`p_unw`	2.297	13.1%	39.9%
mu0=10	`p_wgt`	2.631	12.3%	35.3%
mu0=10	`p_cap`	2.444	11.9%	37.0%
uniform-by-construction	`mse_uni`	1.475	20.3%	54.5%
uniform-by-construction	`p_uni`	1.438	19.8%	55.6%

Same reading rule as §10: compare bar heights within a regime, not across (the natural rows' 83-92% and `mu0=10` 's 34-40% reflect label-range width, not a 2x difference in model quality). Within `mu0=10` the ordering here (`unw` / `cap` , either loss, > `wgt` , either loss, on this aggregate metric) reproduces the same shape as N=3,000 (§9.1/§9.2) — the weighted conditions trade dominant-bin accuracy for tail reach, the rest do the reverse — now confirmed at a second dataset size and across both losses (§9.2's `mse_wgt` ≈ `p_wgt` finding). The `uni` rows top every other condition's within-±1 accuracy at this size (54.5-55.6% vs. the natural factorial's 83-92% — not comparable per the reading rule above, since `uni` 's label range is far narrower — but clearly ahead of `mu0=10` 's 34-40%, which spans a similarly-sized range) — the fullest confirmation in this report that genuine data balance scales better with N than any resampling of imbalanced data does (§7.11, §9.2's note on this size's improvement).

10.2 Dataset composition across every regime and size

`compare_dataset_distributions.py` plots the true `n_parent_pulses` label distribution (share of dataset, not model predictions) for every dataset generated in this project side by side (`dataset_distributions_all.png`):

Dataset	n	Bins present
Natural, N=3,000	3,000	1-6
Natural, N=10,000	10,000	1-8

Dataset	n	Bins present
`mu0=10`, N=3,000	3,000	2-25
`mu0=10`, N=10,000	10,000	1-23
Uniform-by-construction, N=3,000	2,400	1-8 (300/bin)
Uniform-by-construction, N=10,000	6,518 (in progress)	1-7, bin 7 partial

This single figure is the clearest visual summary of the whole investigation's premise: the natural (`mu0=1.16`) bars collapse onto 1-2 pulses and are essentially gone past 6; the `mu0=10` bars form a broad, visibly more symmetric, bell-like hump centered on 9-11; the uniform-by-construction bars are flat by design across whatever bin range was harvested (§7.11, §9) — the only regime where "imbalance" isn't the right word for the label distribution at all. `uniform_10000` finished at 1,000/bin for bins 1-7 (7,000 total, §9's cost diagnostic).

Why is the `mu0=10` distribution so much more symmetric/Gaussian-looking than the natural one, when both are draws from the same kind of process (Poisson(mu0), §2.1)? This is a generic property of the Poisson distribution, not something specific to this pipeline. A Poisson(λ) variable has skewness = $1/\sqrt{\lambda}$ and excess kurtosis = $1/\lambda$ — both shrink as λ grows, so the distribution becomes progressively more symmetric and bell-shaped as its mean increases, converging toward a Gaussian in the large-λ limit (the discrete analogue of the Central Limit Theorem: a Poisson(λ) can be thought of as a sum of λ independent Poisson(1) contributions, and sums of many i.i.d. variables trend Gaussian). At `mu0=1.16` (skewness ≈ 0.93), the distribution is also hard up against its natural lower bound at zero — it cannot go negative, so probability mass piles up at 0-2 with a long thin tail, which reads visually as a sharp spike-and-decay rather than a bell. At `mu0=10` (skewness ≈ 0.32), the mode sits far enough from that zero boundary that the distribution has room to spread symmetrically on both sides, which is exactly what the chart shows. This would happen for any `mu0` pushed similarly far from zero, regardless of the astrophysics — it's arithmetic, not physics.

10.3 A visual gallery: what does a k-pulse light curve actually look like?

`lc_gallery.py` puts one representative light curve per `n_parent_pulses` value side by side, in the same real physical units as `plot_lcs.py` (Time [s] on x, Count rate [cnt/s] on y) with parent pulses marked (`lc_gallery_by_pulse_count.png`, bins 1-8 sourced from `uniform_3000`, 9-16 from the `mu0=10` N=10,000 dataset since it has ample examples at those counts). Two things stand out immediately: pulses are only individually countable by eye up to roughly 3-4 — beyond that, later pulses increasingly ride on the decay tail of earlier ones and overlap into a single ragged-looking envelope, which is a visual demonstration of exactly why the CNN's accuracy craters past ~5-6 pulses (§7.2, §2.8); and there is no obvious *duration* trend with pulse count — a 16-pulse light curve isn't visibly "longer" than a 3-pulse one in this gallery, since pulse count and pulse timing/width are governed by separate, independent parameters in the SS96 model (`mu0` vs. `tau_min` / `tau_max` / `delta1` / `delta2`, §2.1) — so pulse count has to be read from the internal structure of the burst, not from how long it lasts. This gallery is a purely visual reference for eyeballing rough morphological similarity against a real burst; per §8, it is not a validation method, since no real burst comes with a known true pulse count to check against.

10.4 Which condition was actually best, per dataset — and why

"Best" has to mean *best at engaging classes beyond the dominant mode*, not highest aggregate exact-match/within-±1 — §7.2 and §10's reading rule both establish that the unweighted/capped-no-op conditions top those metrics purely by exploiting however concentrated the label distribution is, which is the mode-collapse problem this whole investigation exists to diagnose, not a sign of a better model.

Dataset	Best condition	Why
μ0 = 1.16, N=3,000	`mse_wgt`	Balanced sampler reaches classes 3-4 no other condition does (21.9%/4.3%, §7.11) — the sampler, not the loss, is doing the real work; MSE unexpectedly beats Poisson once resampling is already correcting the imbalance (§7.11's unresolved finding).
μ0 = 1.16, N=10,000	`mse_wgt` (best rare-class reach) or `p_cap` / `mse_cap` (best redistribution among classes 1-3 once three are tied at parity, §7.10)	Same mechanism as N=3,000, confirmed at scale — `mse_wgt` reaches further into the tail (class 3: 28.6%), while capping only helps once enough classes land at exactly the same size.
μ0 = 10, N=3,000	`p_wgt`	Every other condition (`unw` / `cap` , either loss) collapses onto the natural mode at 9-10 with 0% on classes below 6; `p_wgt` is the only one that reaches 6, 7, 12, 13, 14 at all (§9.1). `mse_cap` / `p_cap` are literal no-ops here (max class = 374, under the 1,000 cap).
μ0 = 10, N=10,000	`p_wgt` (barely — `mse_wgt` is now nearly as good)	With all 6 conditions complete (§9.2), `mse_wgt` reaches almost the same tail bins as `p_wgt` (4, 5, 13, 14, 15 vs. 5, 6, 13, 14, 15, 16) — the weighted-sampler effect isn't loss-specific here either. `p_wgt` keeps a modest aggregate edge (within-±1 35.3% vs. 33.9%), so it's still the pick, but the margin is much narrower than the natural-data comparisons. `p_cap` / `mse_cap` redistribute only *within* the already-populous 8-11 core, since capping can't touch classes that were never oversized to begin with.
Uniform-by-construction, N=3,000	`p_uni`	Marginally beats `mse_uni` on every metric (MAE 1.731 vs. 1.861, within-±1 45.8% vs. 40.6%, §7.11) — the one place in the whole investigation Poisson loss's theoretical edge (matching count-data noise, §2.5) actually shows up, plausibly because there's no competing resampling mechanism fighting it here; the data is already flat, so the loss function is the only lever left.
Uniform-by-construction, N=10,000	`p_uni` (by a hair)	Nearly tied with `mse_uni` (MAE 1.438 vs. 1.475, within-±1 55.6% vs. 54.5%; `mse_uni` edges ahead only on exact-match, 20.3% vs. 19.8%) — unlike at N=3,000, where `p_uni` won clearly on every metric. More data closes most of the loss-function gap once the data itself is already balanced (§7.11).

The six rows above plotted as one per-class accuracy chart (`compare_best_per_dataset.png`) and the same runs' aggregate metrics side by side:

Dataset	Best condition	Test MAE	Exact-match	Within-±1
μ0 = 1.16, N=3,000	`mse_wgt`	0.758	34.4%	91.3%
μ0 = 1.16, N=10,000	`mse_wgt`	0.801	35.9%	86.5%
μ0 = 10, N=3,000	`p_wgt`	2.793	10.4%	32.4%
μ0 = 10, N=10,000	`p_wgt`	2.631	12.3%	35.3%
Uniform, N=3,000	`p`	1.731	15.0%	45.8%
Uniform, N=10,000	`p`	1.438	19.8%	55.6%

Same reading rule as §10 and §10.1: within-±1 isn't comparable straight down this column either — natural's 86-91% reflects its narrow 1-6/1-8 label range, not better modeling than μ0 = 10's 32-35% over a 2-25 range. Compare each row against its *own* history (does 3k→10k improve it?), not against the other rows.

The pattern across every regime and size: **reweighting existing rare examples (`wgt`) consistently reaches further into the tail than any resampling of the same underlying data (`cap`), and capping only ever helps as a side effect of accidentally tying several classes at exact parity — never by rescuing classes that were rare to begin with.** The one case where none of this matters is when the data isn't imbalanced in the first place (`uni`), where the question resolves to loss function alone rather than sampler strategy.

Just creating synthetic LCs — no GA involved anywhere in this pipeline.

simulate_GRBs.py (which is what's generating the 10k/20k datasets) only draws light curves from the SS96 avalanche model using the 7 parameters already sitting in config.ini — the ones from the paper's already-completed GA fit (mu=1.34, mu0=1.16, alpha=2.53, delta1=-0.75, delta2=0.27, tau_min=0.03, tau_max=56.8). It never touches geneticgrbs.py (the actual GA optimizer, which searches for those parameter values by fitting against the real Swift/BAT catalog) — that would additionally need the real catalog data, which we established early on isn't available on this machine and isn't needed for what we're doing.

So the full chain is: GA fit already done by the original authors (real data → 7 numbers, not run by us) → our pipeline draws synthetic LCs from those fixed 7 numbers → CNN trained to predict n_parent_pulses from those synthetic LCs. This is exactly the distinction covered in REPORT.md §7.4 (why real Swift/BAT LCs should statistically resemble ours despite never touching the catalog ourselves).